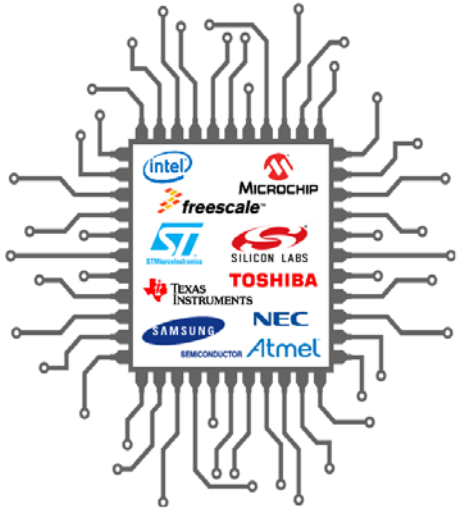
I/O Port Programming





Thareendhra Keerthi Wijayasiriwardhane

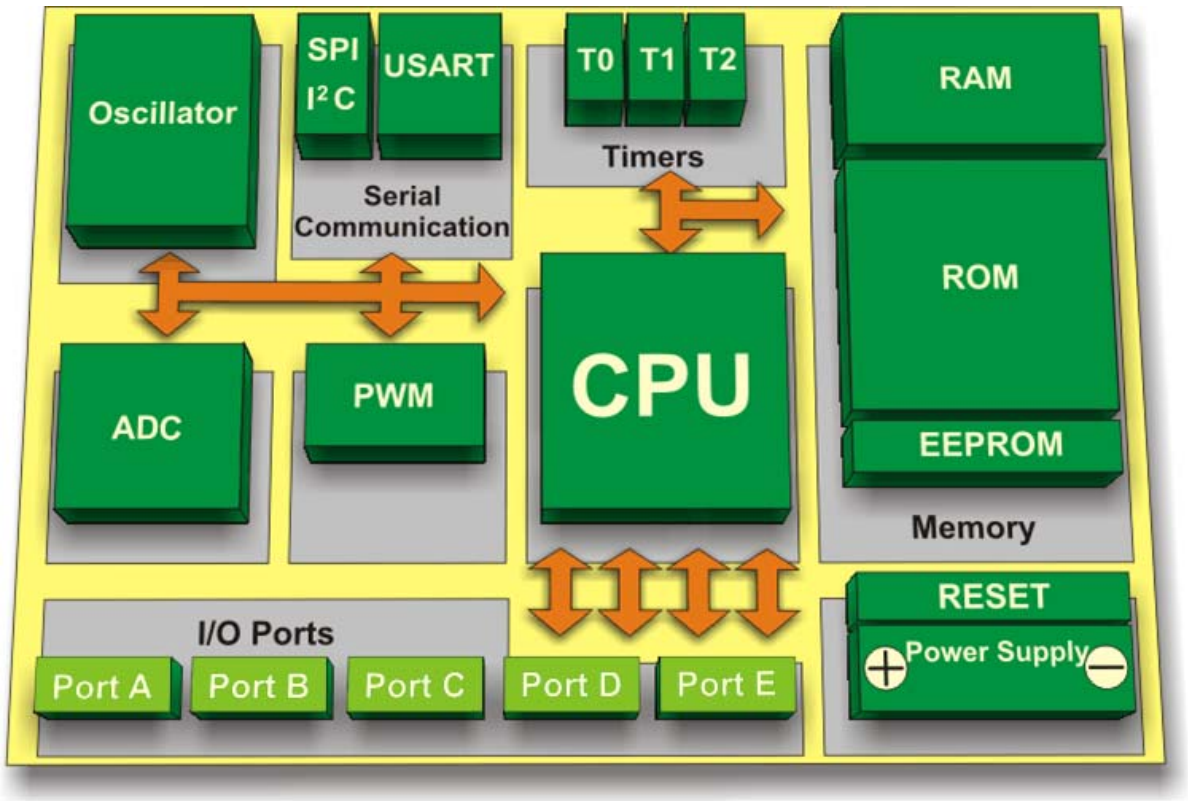
Department of Industrial Management
Faculty of Science
University of Kelaniya
thareen@kln.ac.lk

Learning Outcomes

- On completion of this session, you should be able to
  - Explain the functionality of I/O registers
  - Construct a microcontroller-based system and program it to drive an actuator
  - Compare hardwired vs. software-based approaches
  - Discuss the pros and cons of different firmware implementations
  - Construct a microcontroller-based system and program it to read a digital input and then drive an actuator

I/O Ports

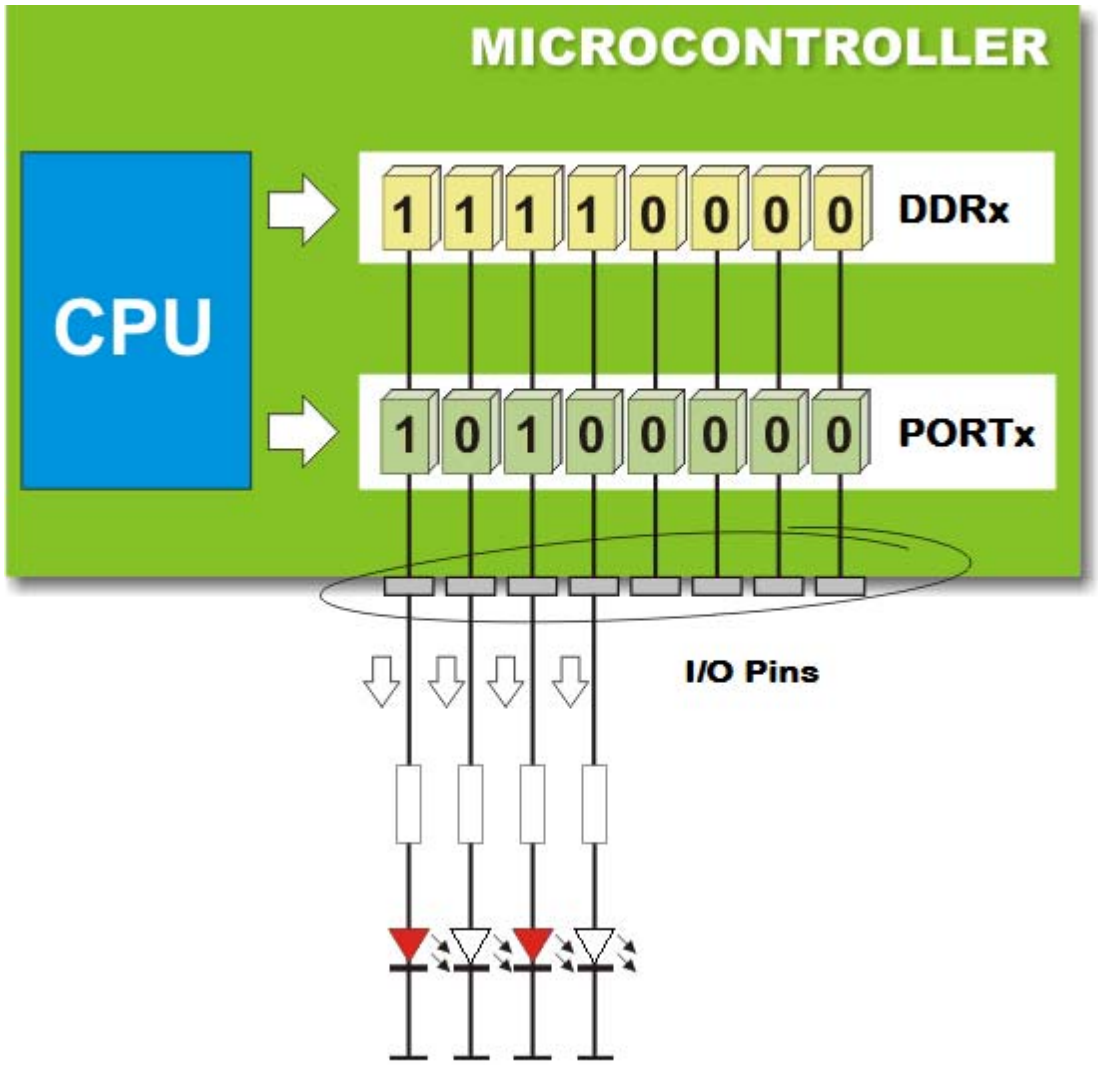
- Used to get the state of physical world with sensors
- Also used to drive actuators and thereby change the state of physical world



I/O Registers

- Registers associated with an I/O Port
- Data Direction Register (DDR)
  - Determines data direction
  - Set to configure port pins as outputs
  - Cleared to configure port pins as inputs
- Port Output RegisTer (PORT)
  - Used to specify the state of port pins

DDR and PORT Registers

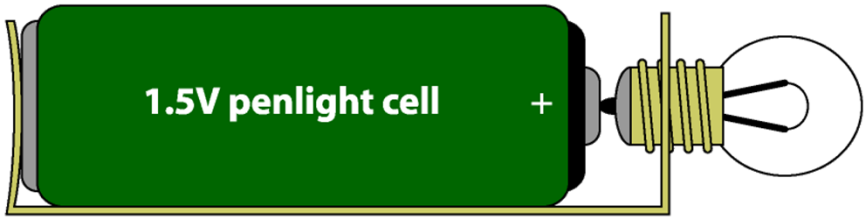


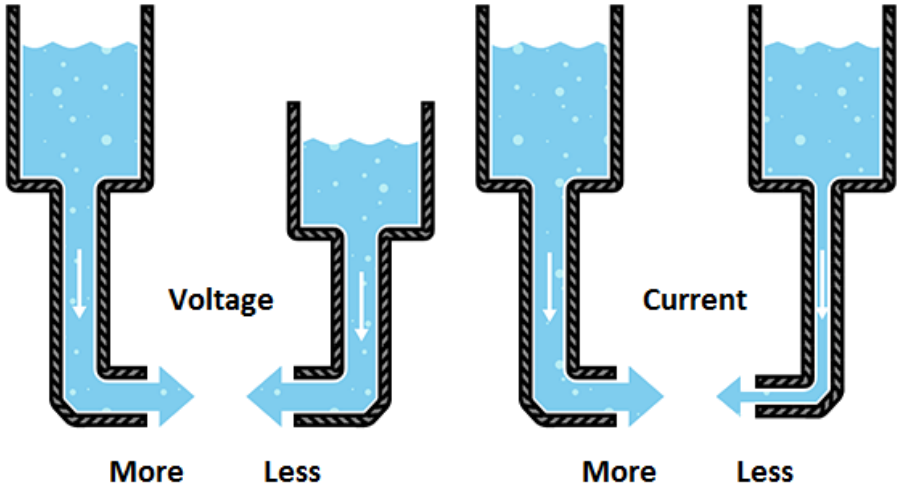
System 1

blinkerAsm - Blinks an LED every ½ a second using Assembly

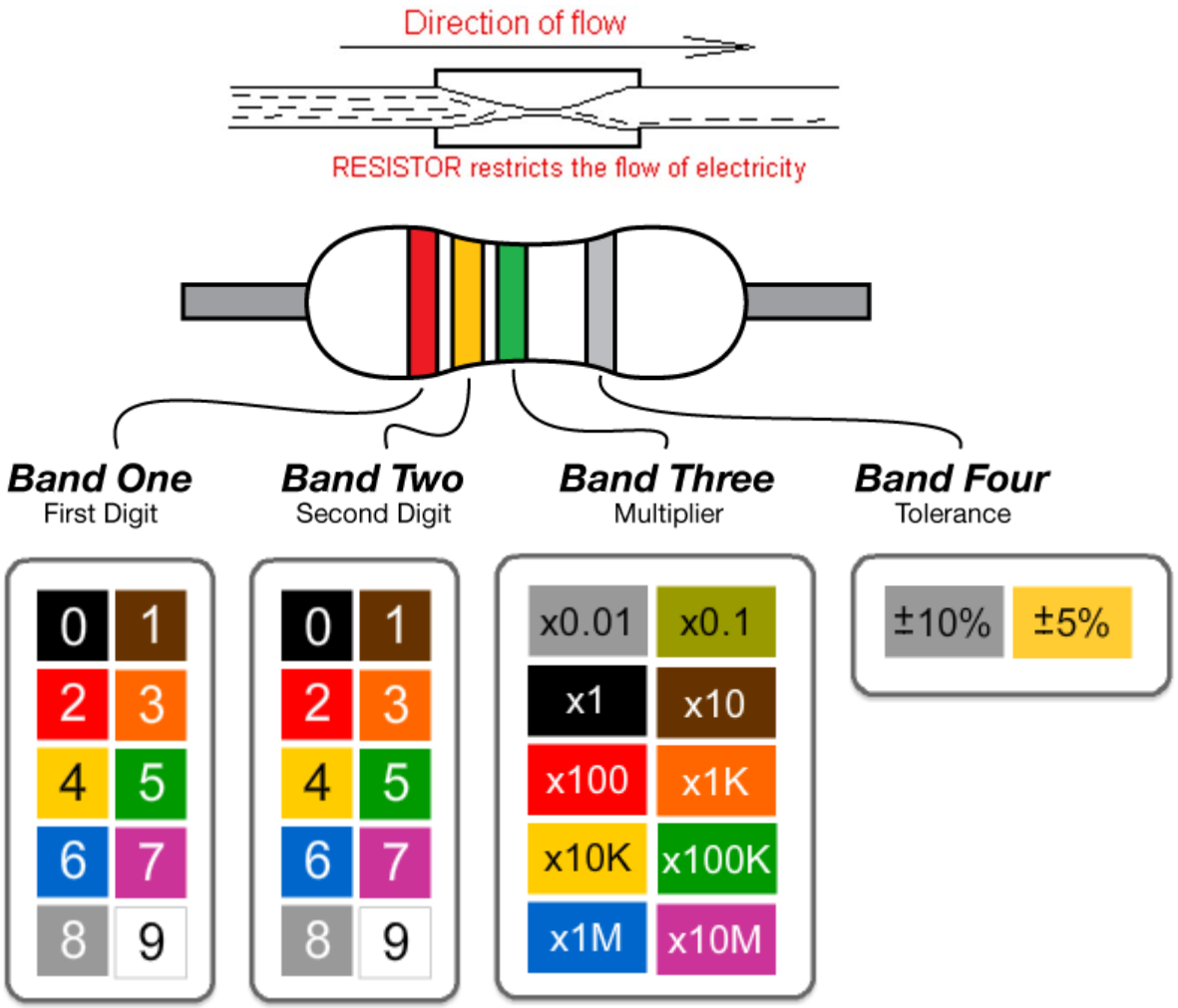
Electricity

- Current
- Voltage

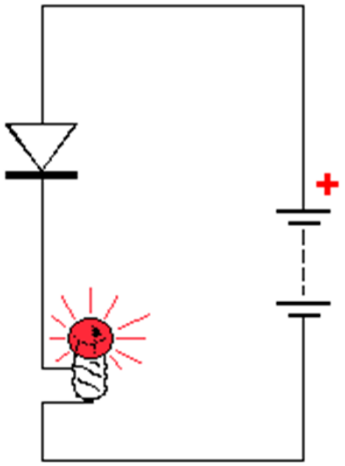


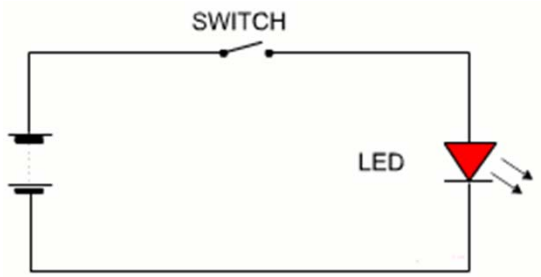


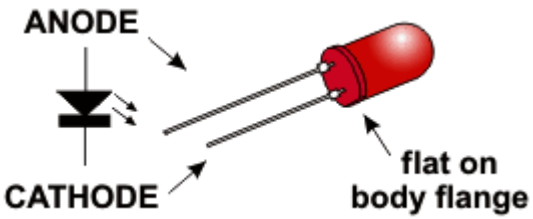
Resistor

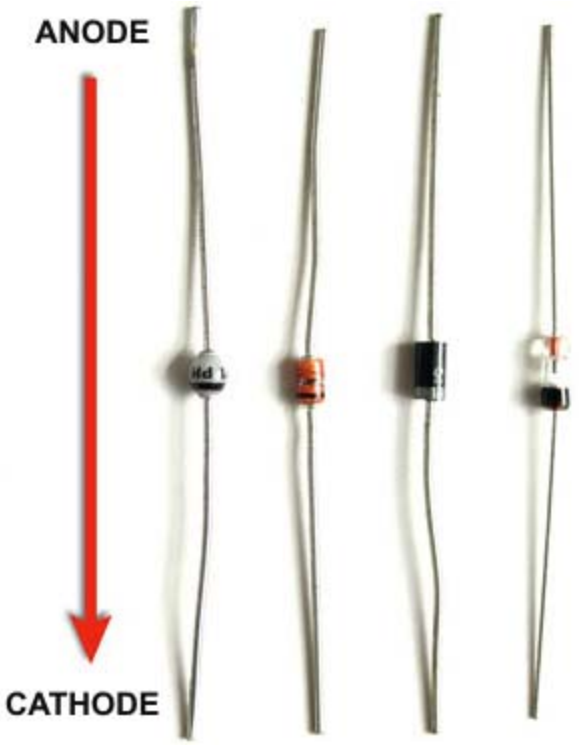

Direction of flow
RESISTOR restricts the flow of electricity
Band One
First Digit
Band Two
Second Digit
Band Three
Multiplier
Band Four
Tolerance
0
1
2
3
4
5
6
7
8
9
x0.01
x0.1
x1
x10
x100
x1K
x10K
x100K
x1M
x10M
±10%
±5%

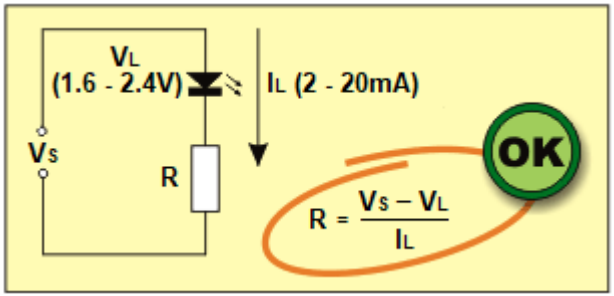
Diode


+

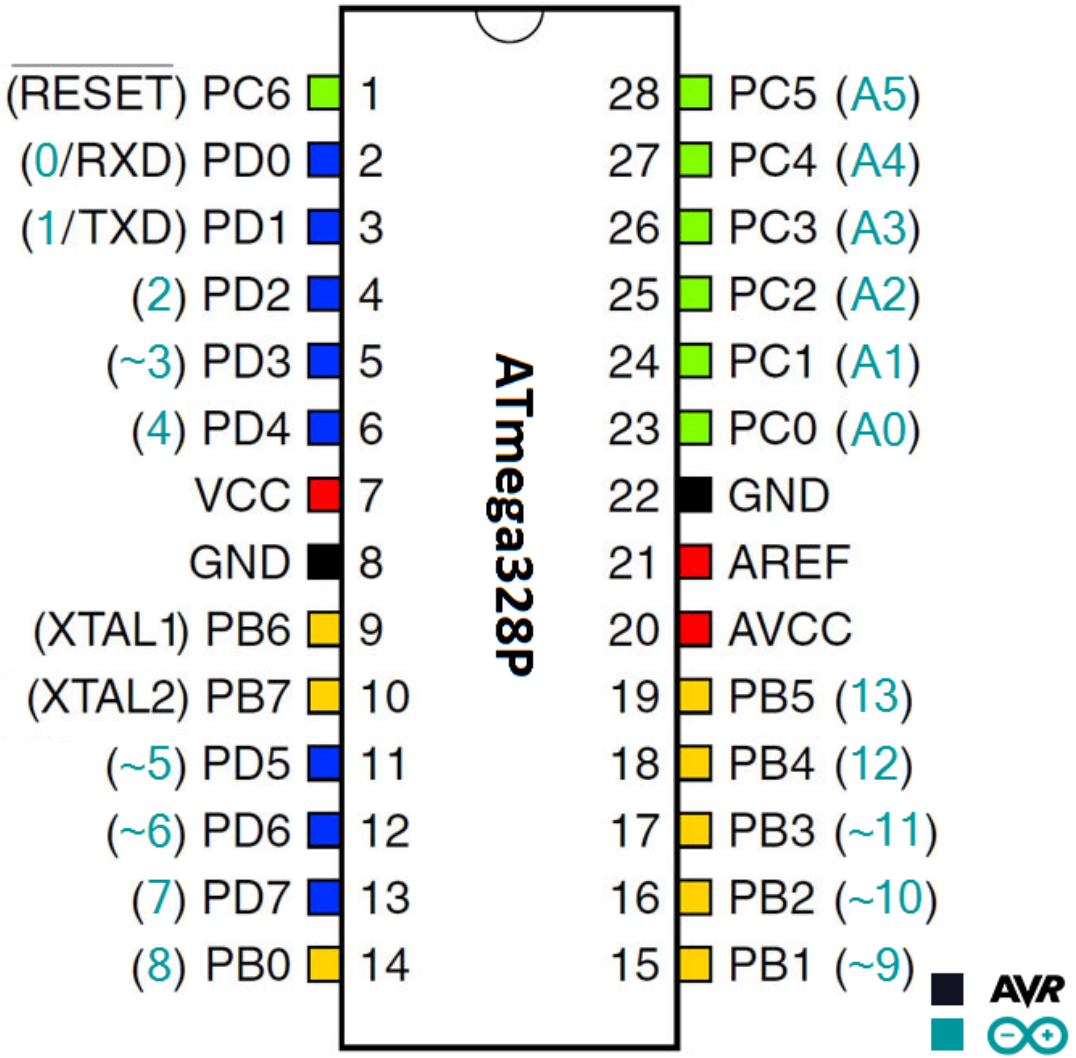

SWITCH
LED


ANODE
CATHODE
flat on
body flange


ANODE
CATHODE

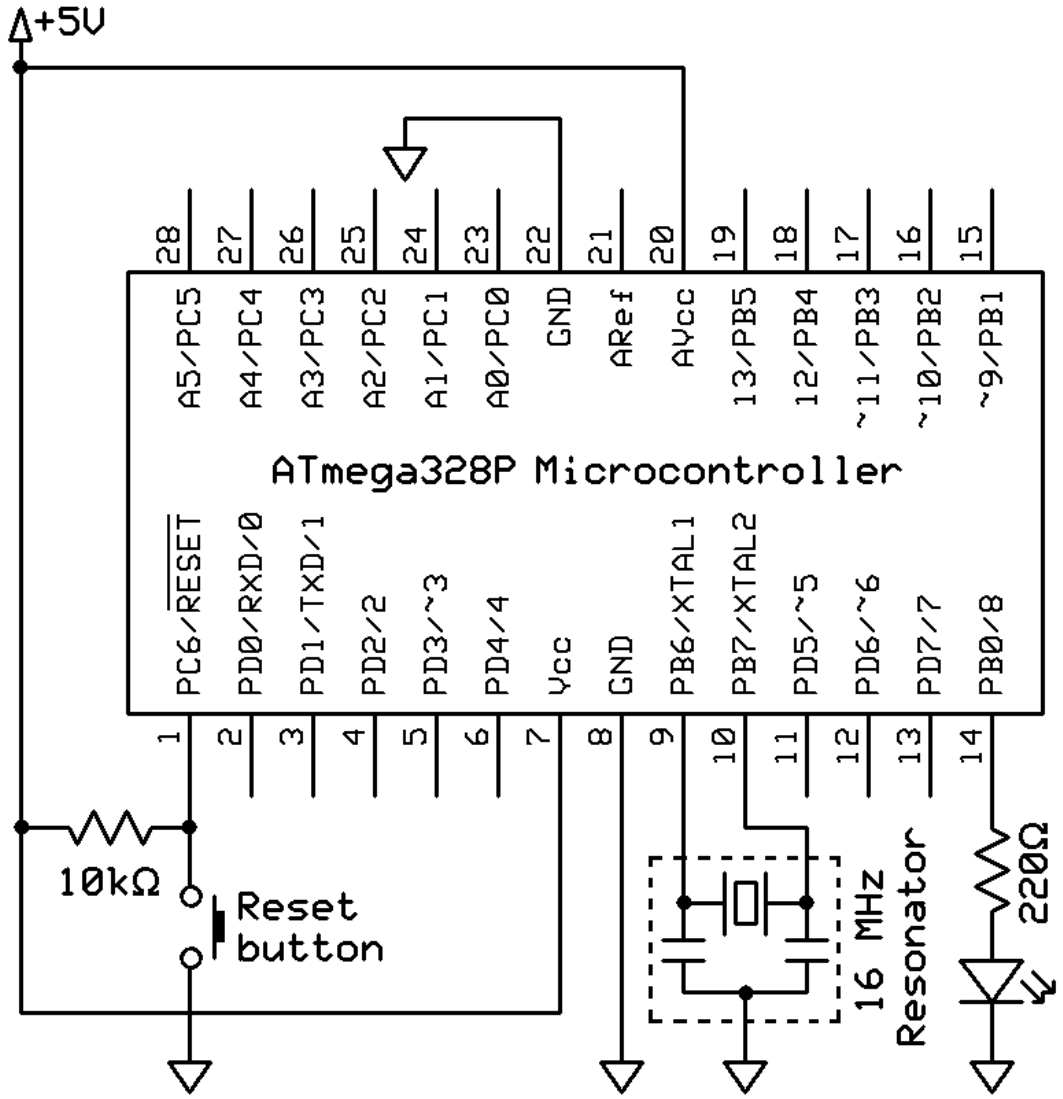

VL
(1.6 - 2.4V)
IL (2 - 20mA)
VS
R
OK
R = VS – VL
IL

ATmega328P I/O Ports

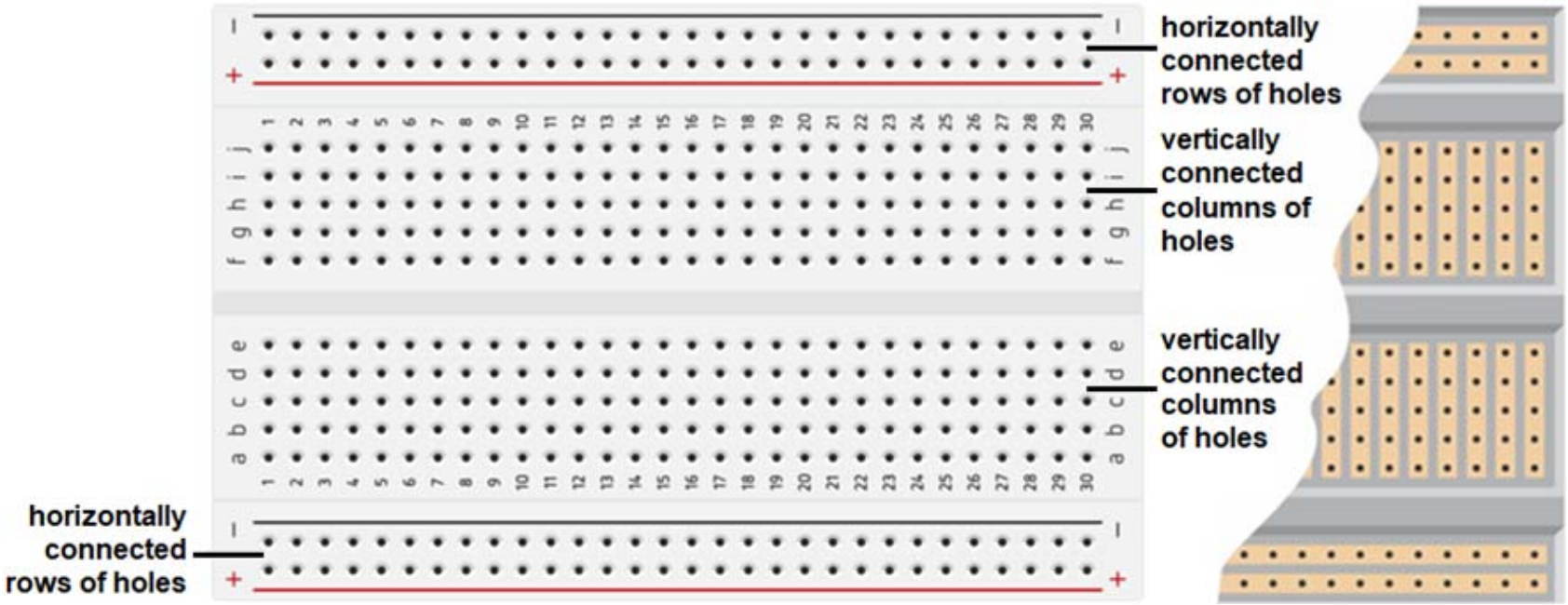


Hardware

- Schematic Diagram



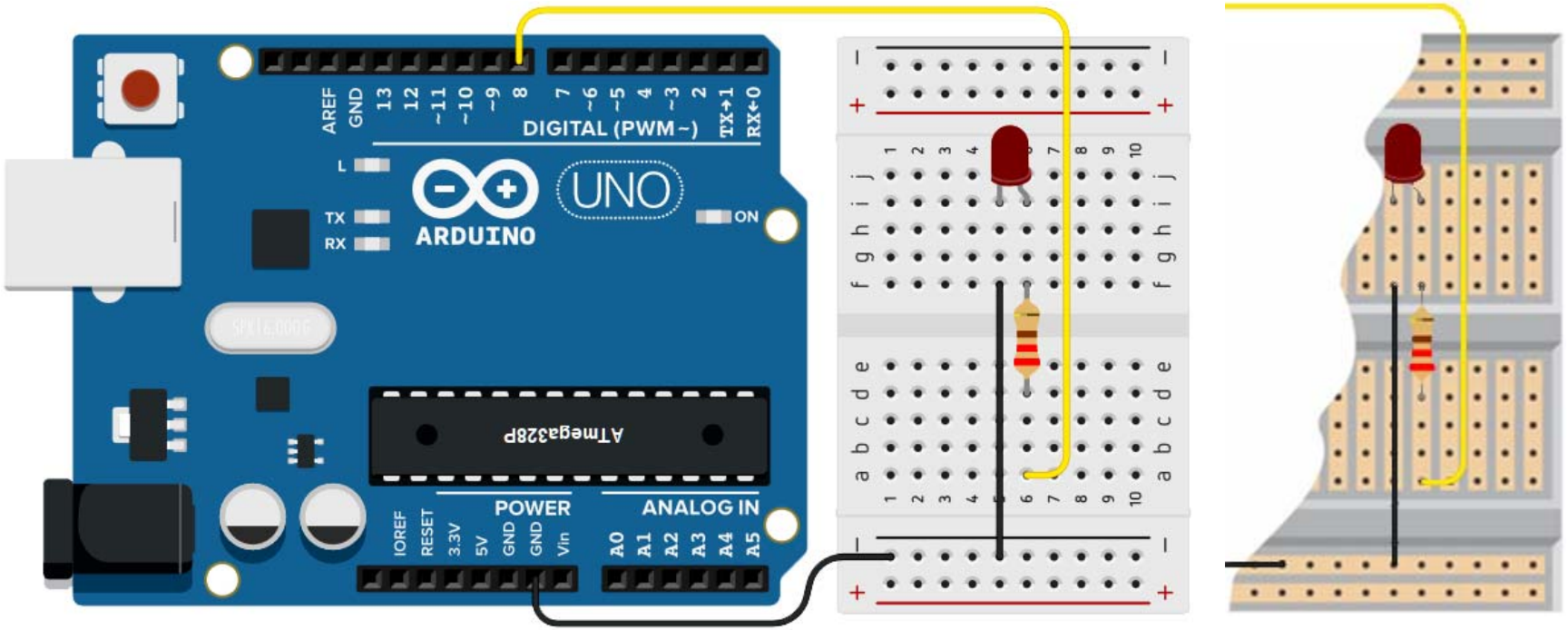
Breadboard


horizontally connected rows of holes
vertically connected columns of holes
vertically connected columns of holes
horizontally connected rows of holes



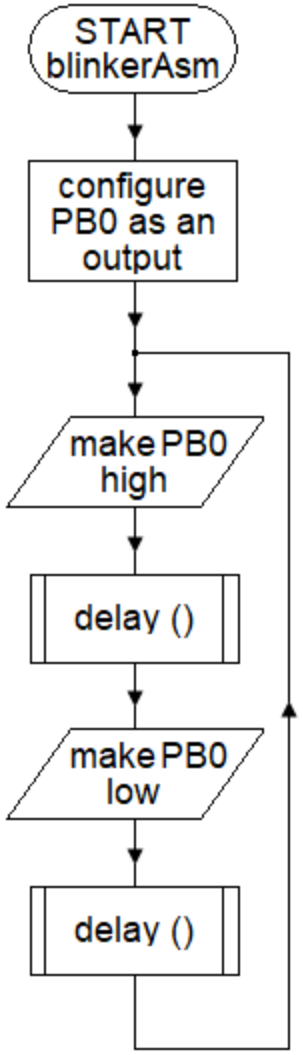
Hardware

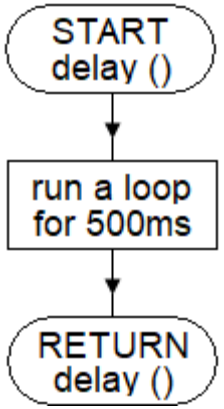
- Wiring Diagram



Firmware

- Algorithm

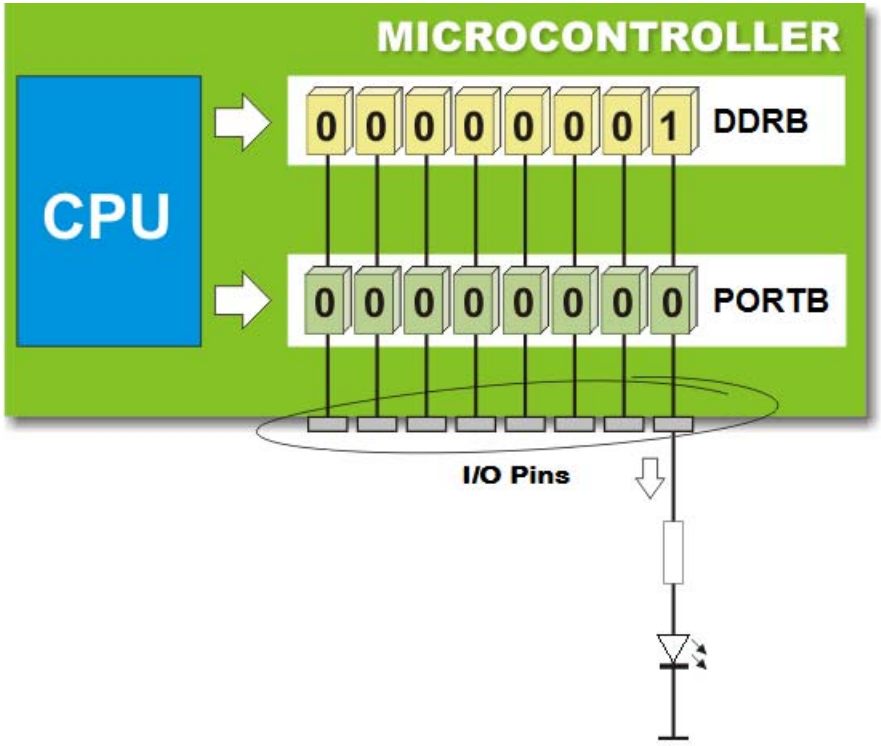




Configuring an I/O Pin

- Configure Pin 0 of Port B as an output
  - Use Set Bit in I/o register instruction

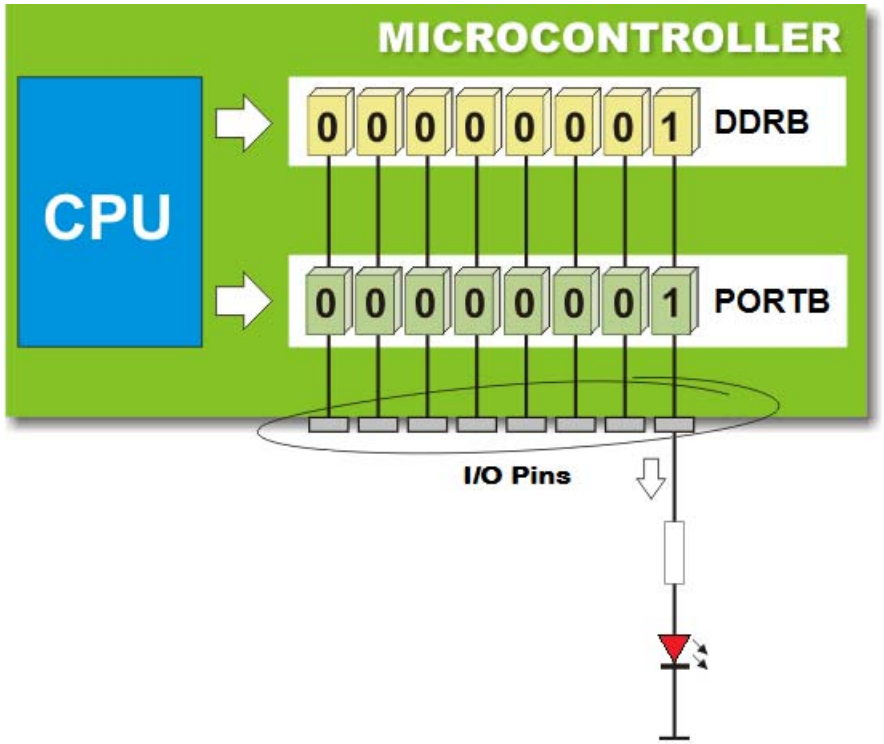
```
SBI DDRB, 0 ; set bit 0 of DDRB & configure PB0 as an output
```



Making an I/O Pin High

- Change the state of Pin 0 of Port B high
  - Use Set Bit in I/o register instruction

```
SBI PORTB, 0    ; set bit 0 of PORTB & turn on LED
```



Delaying

- Engage CPU by running a loop and delay half a second
  - Use LoaD Immediate, DECrement and BRanch if Not Equal instructions

```
        LDI R16, 41     ; load 41 to R16
LOOP1:  LDI R17, 255    ; load 255 to R17
LOOP2:  LDI R18, 255    ; load 255 to R18
LOOP3:  DEC R18         ; decrement R18 by 1
        BRNE LOOP3      ; branch to LOOP3 if result is not 0
        DEC R17         ; decrement R17 by 1
        BRNE LOOP2      ; branch to LOOP2 if result is not 0
        DEC R16         ; decrement R16 by 1
        BRNE LOOP1      ; branch to LOOP1 if result is not 0
```

Delay Calculation

- Calculate clock cycle time

$$\text{clock cycle time } (T) = \frac{1}{\text{frequency}}$$

- Calculate number of clock cycles starting from the most inner loop

```
      LDI R16, X
LOOP1:LDI R17, Y
LOOP2:LDI R18, Z
LOOP3:DEC R18
      BRNE LOOP3
      DEC R17
      BRNE LOOP2
      DEC R16
      BRNE LOOP1
```

C, B, A

$$C=(1+2)x(Z-1)+(1+1)x1$$

$$B=(1+C+1+2)x(Y-1)+(1+C+1+1)x1$$

$$A=(1+B+1+2)x(X-1)+(1+B+1+1)x1$$

$$\text{clock cycles } (N) = 1 + A$$

$$\text{delay} = T \times N$$

Calling a Subroutine

- Execute the instructions for delaying half a second as a subroutine
  - Use CALL instruction

```
CALL DELAY              ; call DELAY subroutine
```

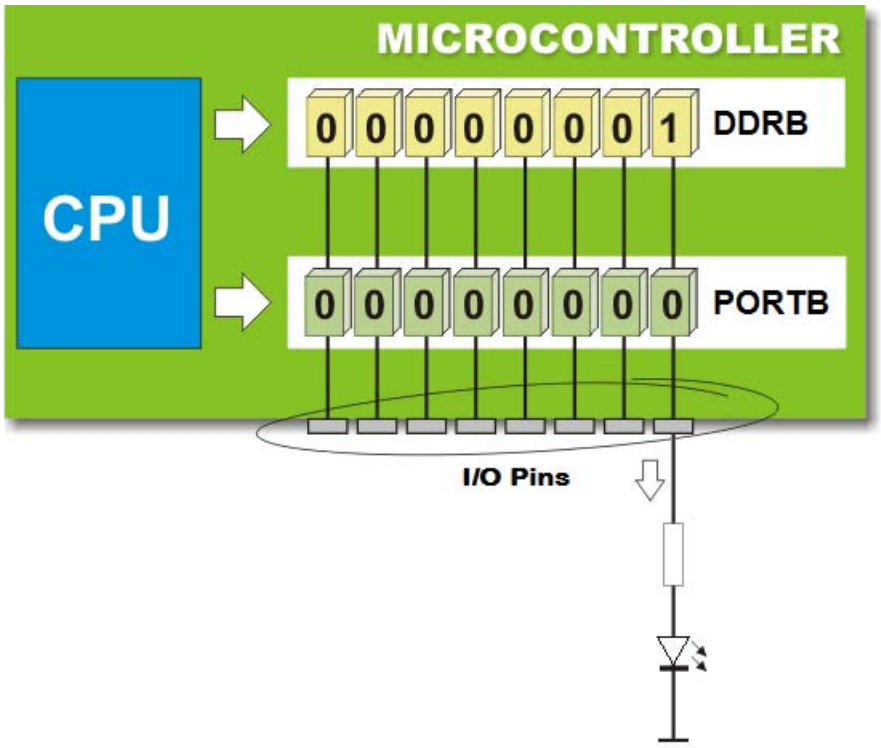
  - Use RETurn instruction to return from subroutine

```
DELAY:  LDI R16, 41     ; load 41 to R16
LOOP1:  LDI R17, 255    ; load 255 to R17
LOOP2:  LDI R18, 255    ; load 255 to R18
LOOP3:  DEC R18         ; decrement R18 by 1
        BRNE LOOP3      ; branch to LOOP3 if result is not 0
        DEC R17         ; decrement R17 by 1
        BRNE LOOP2      ; branch to LOOP2 if result is not 0
        DEC R16         ; decrement R16 by 1
        BRNE LOOP1      ; branch to LOOP1 if result is not 0
        RET             ; return to calling point
```

Making an I/O Pin Low

- Change the state of Pin 0 of Port B low
  - Use Clear Bit in I/o register instruction

```
CBI PORTB, 0     ; clear bit 0 of PORTB & turn off LED
```



Performing Tasks Indefinitely

- Perform the tasks such as change the state of Pin 0 of Port B high, delay 500ms, change the state of Pin 0 of Port B low and again delay 500ms, indefinitely
  - Use JuMP instruction

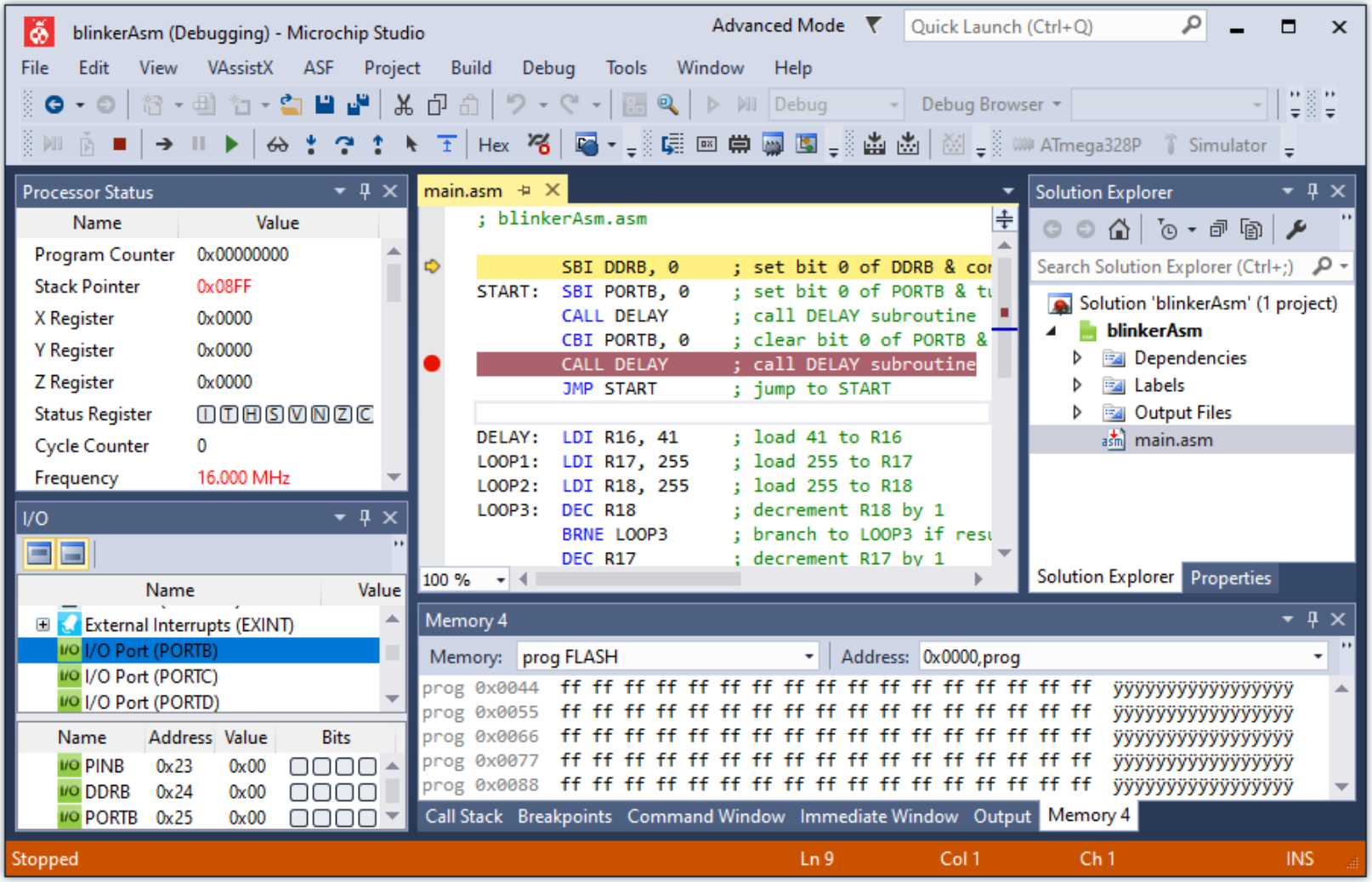
```
START:  SBI PORTB, 0    ; set bit 0 of PORTB & turn on LED
        CALL DELAY      ; call DELAY subroutine
        CBI PORTB, 0    ; clear bit 0 of PORTB & turn off LED
        CALL DELAY      ; call DELAY subroutine
        JMP START       ; jump to START
```

  - Infinite loop

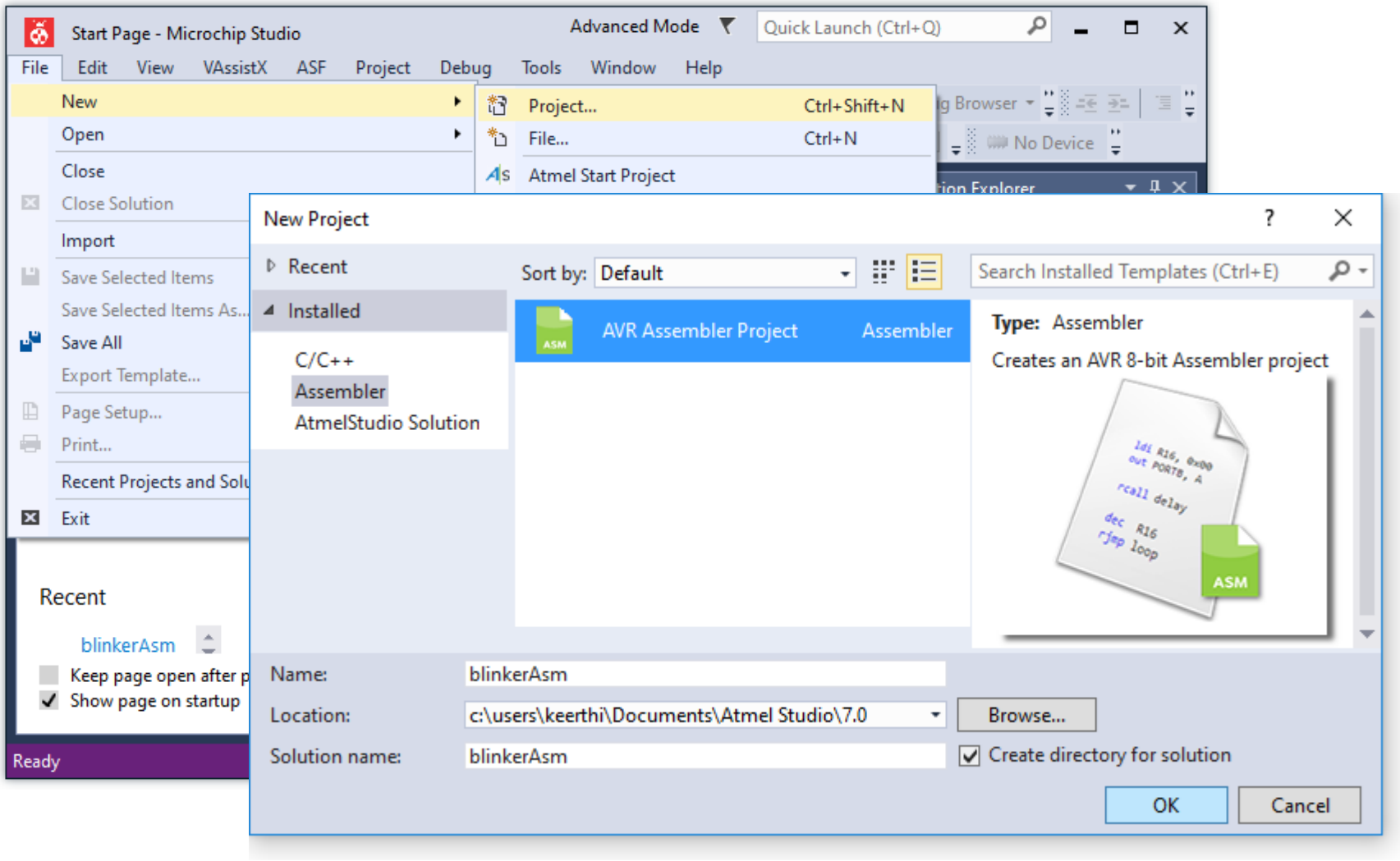
Assembly Program

```
1   ; blinkerAsm.asm
2
3           SBI DDRB, 0     ; set bit 0 of DDRB & configure PB0 as an output
4   START:  SBI PORTB, 0    ; set bit 0 of PORTB & turn on LED
5           CALL DELAY      ; call DELAY subroutine
6           CBI PORTB, 0    ; clear bit 0 of PORTB & turn off LED
7           CALL DELAY      ; call DELAY subroutine
8           JMP START       ; jump to START
9
10  DELAY:  LDI R16, 41     ; load 41 to R16
11  LOOP1:  LDI R17, 255    ; load 255 to R17
12  LOOP2:  LDI R18, 255    ; load 255 to R18
13  LOOP3:  DEC R18         ; decrement R18 by 1
14          BRNE LOOP3      ; branch to LOOP3 if result is not 0
15          DEC R17         ; decrement R17 by 1
16          BRNE LOOP2      ; branch to LOOP2 if result is not 0
17          DEC R16         ; decrement R16 by 1
18          BRNE LOOP1      ; branch to LOOP1 if result is not 0
19          RET             ; return to calling point
```

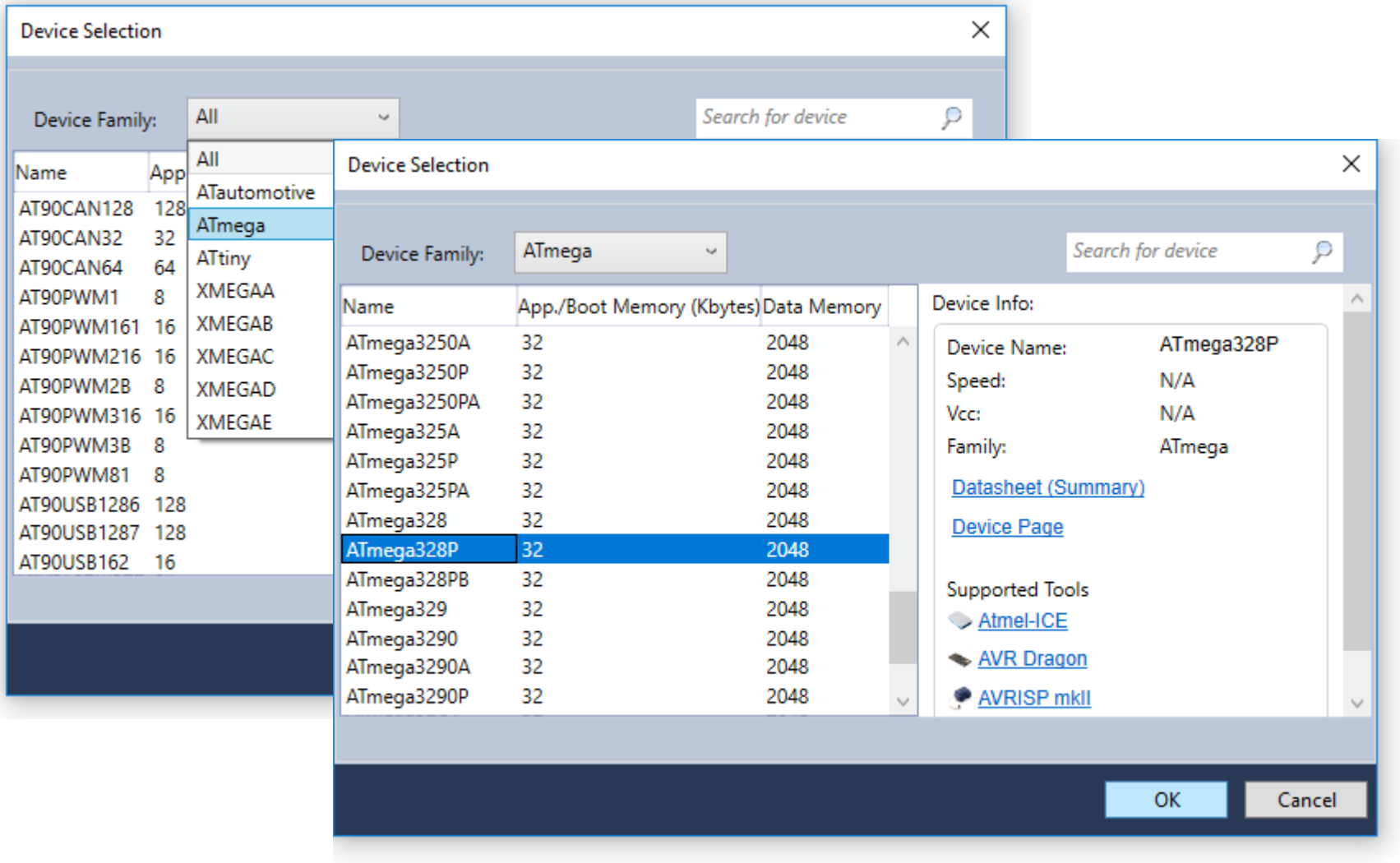
Microchip Studio



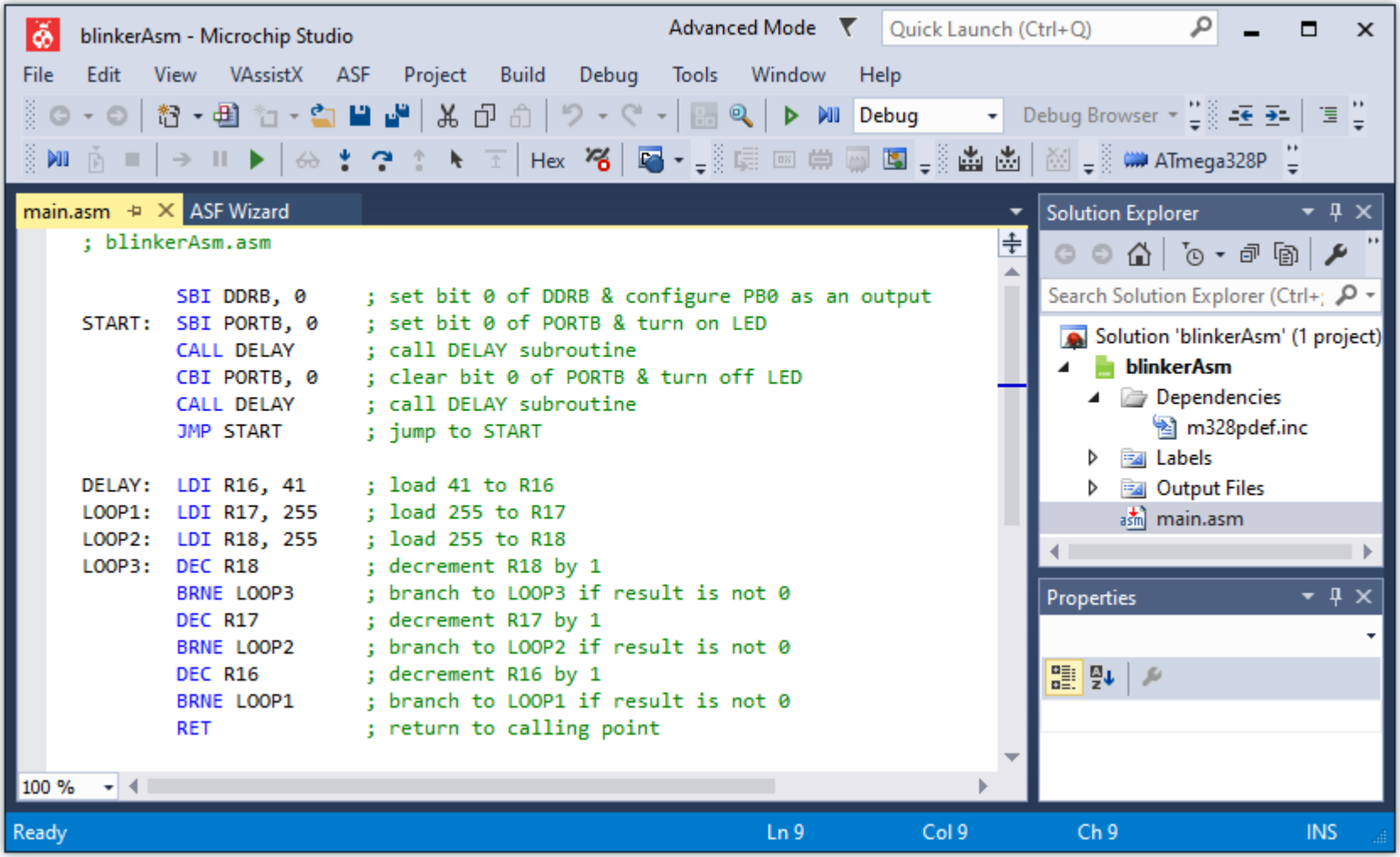
Creating an Assembler Project



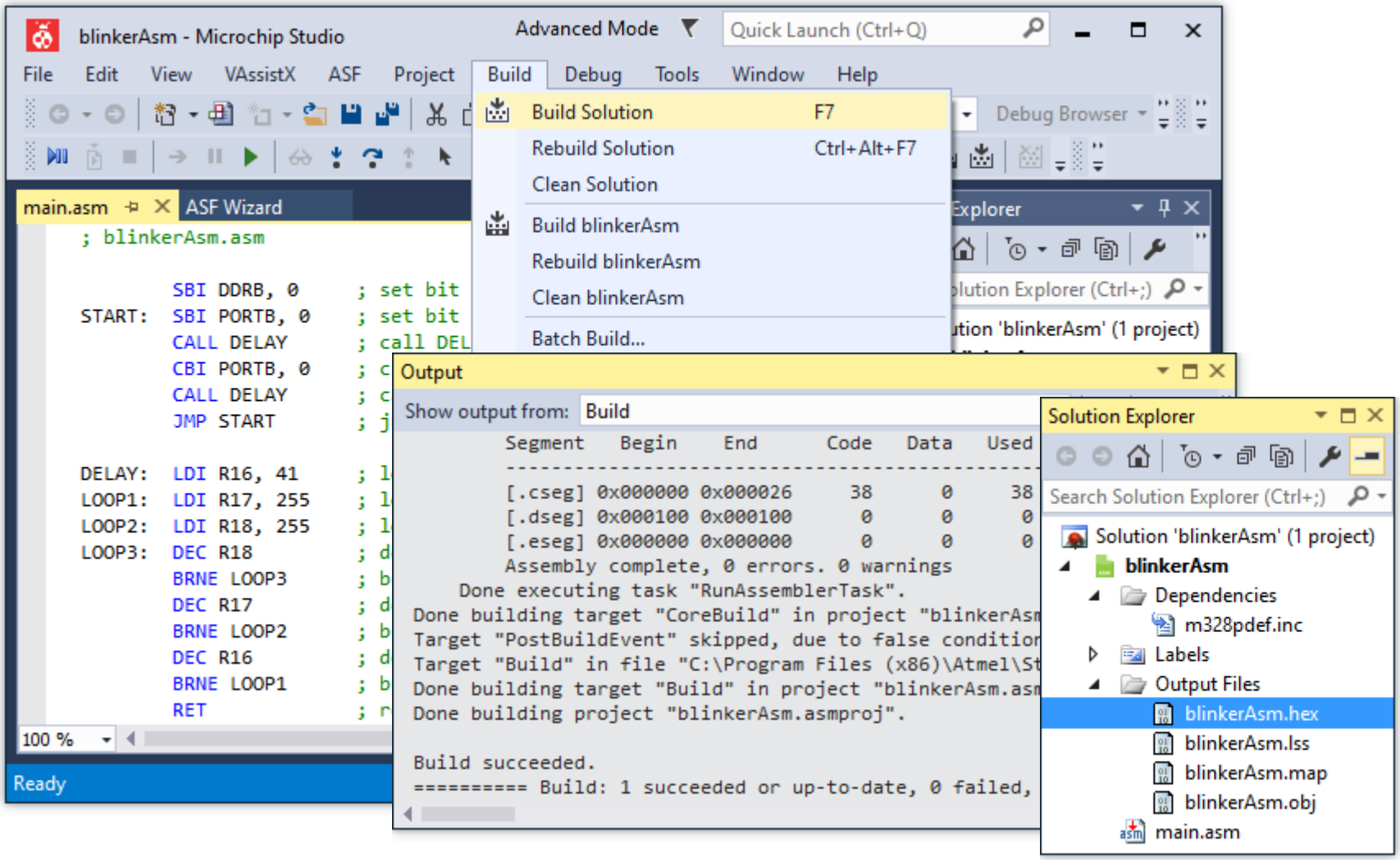
Device Selection



Assembly Program

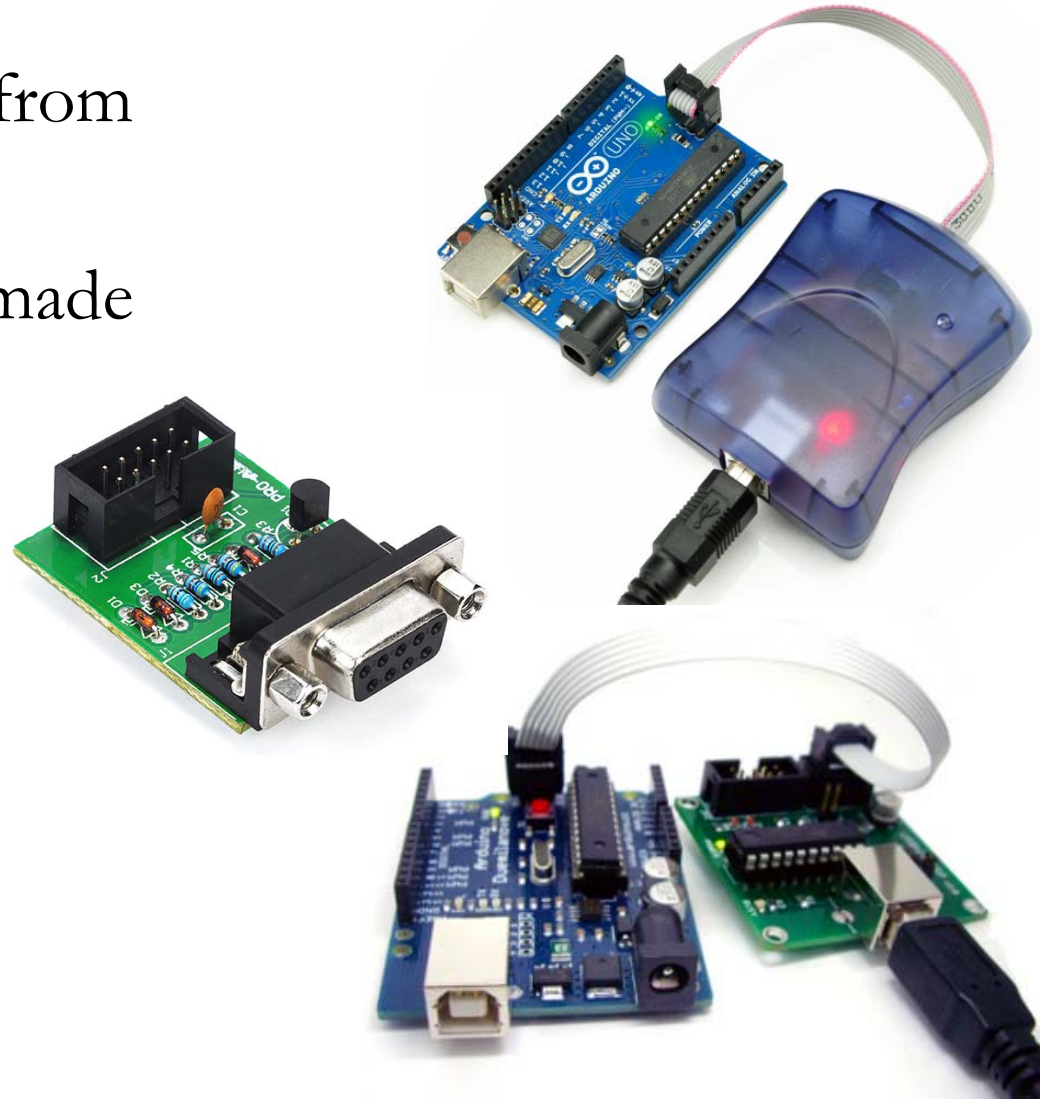


Building HEX File



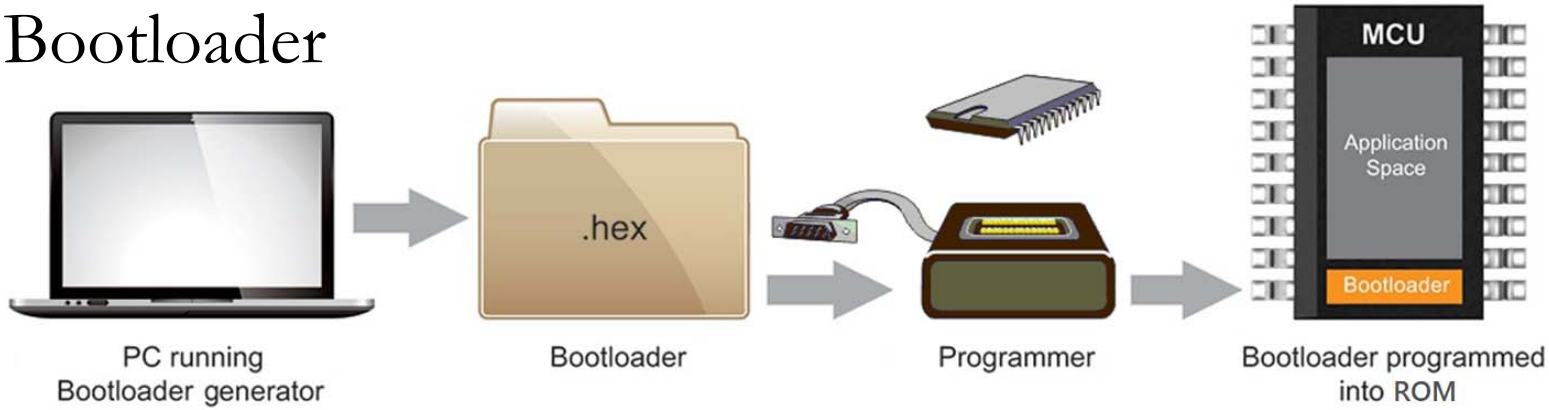
Programming Options

- COTS programmer from manufacturer
- Aftermarket/home made programmers
  - USB
  - Serial
- Self-programming

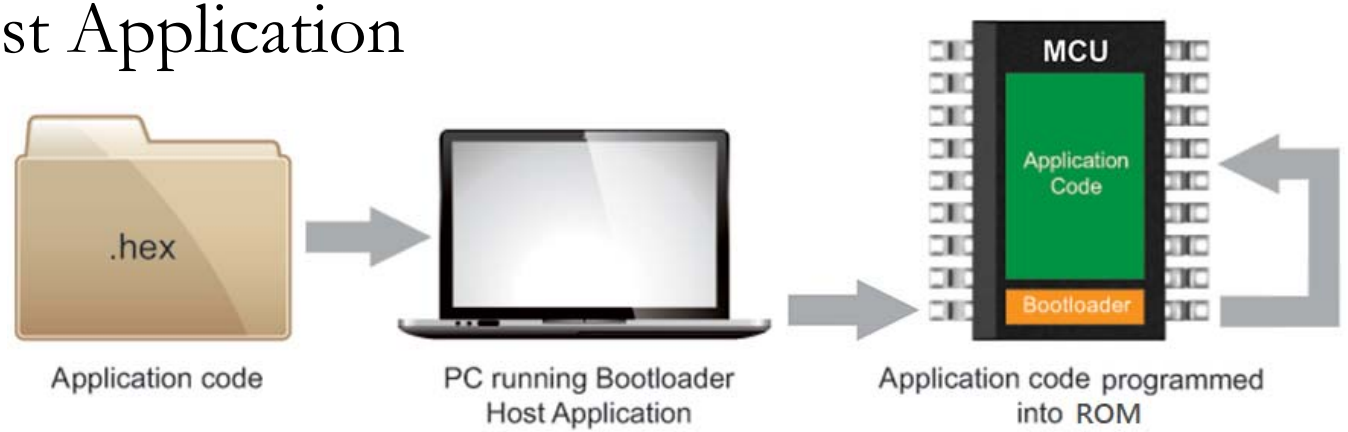


Self-Programming

- Bootloader

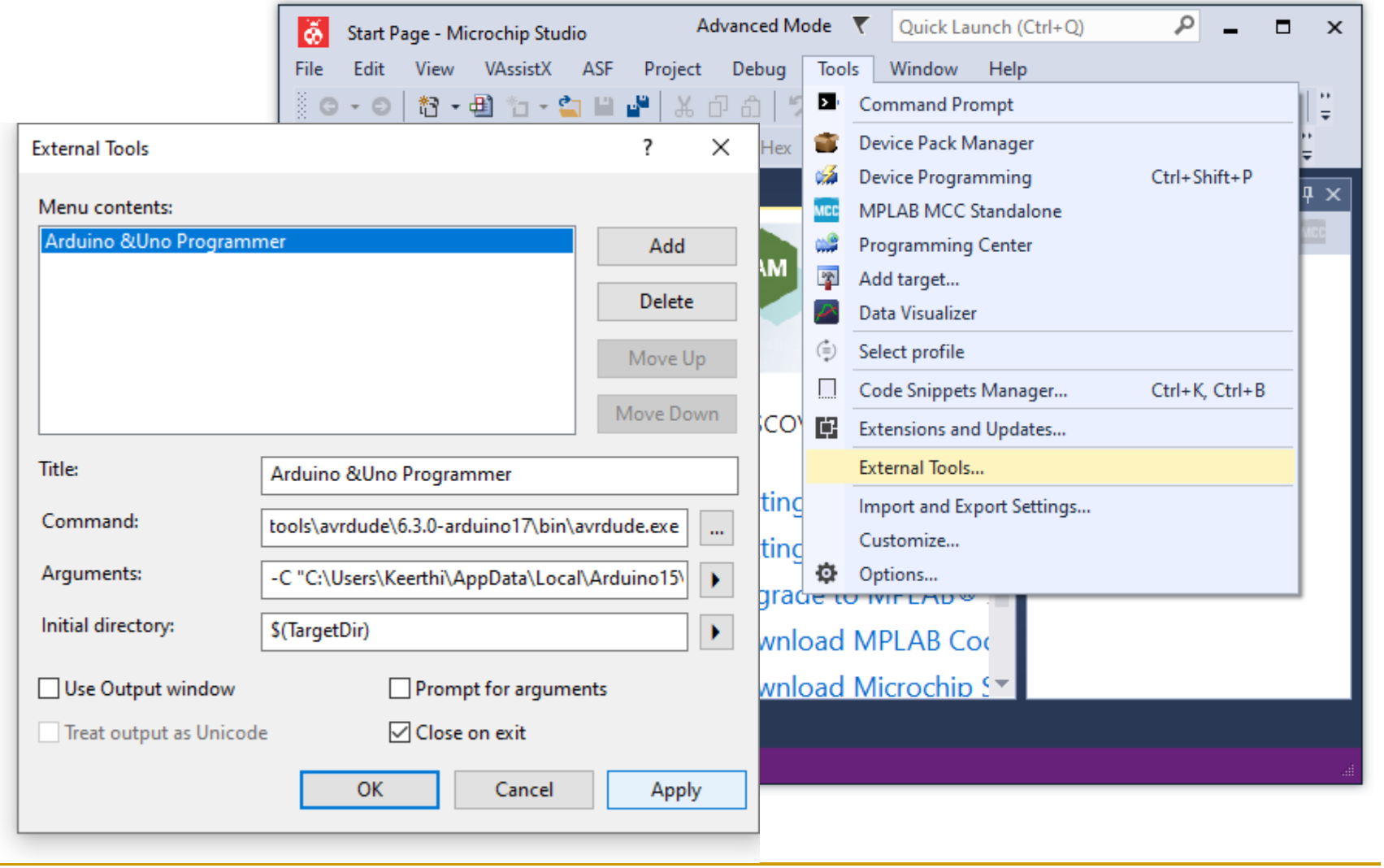


- Host Application



  - AVRDUDE (AVR Downloader UploaDEr)

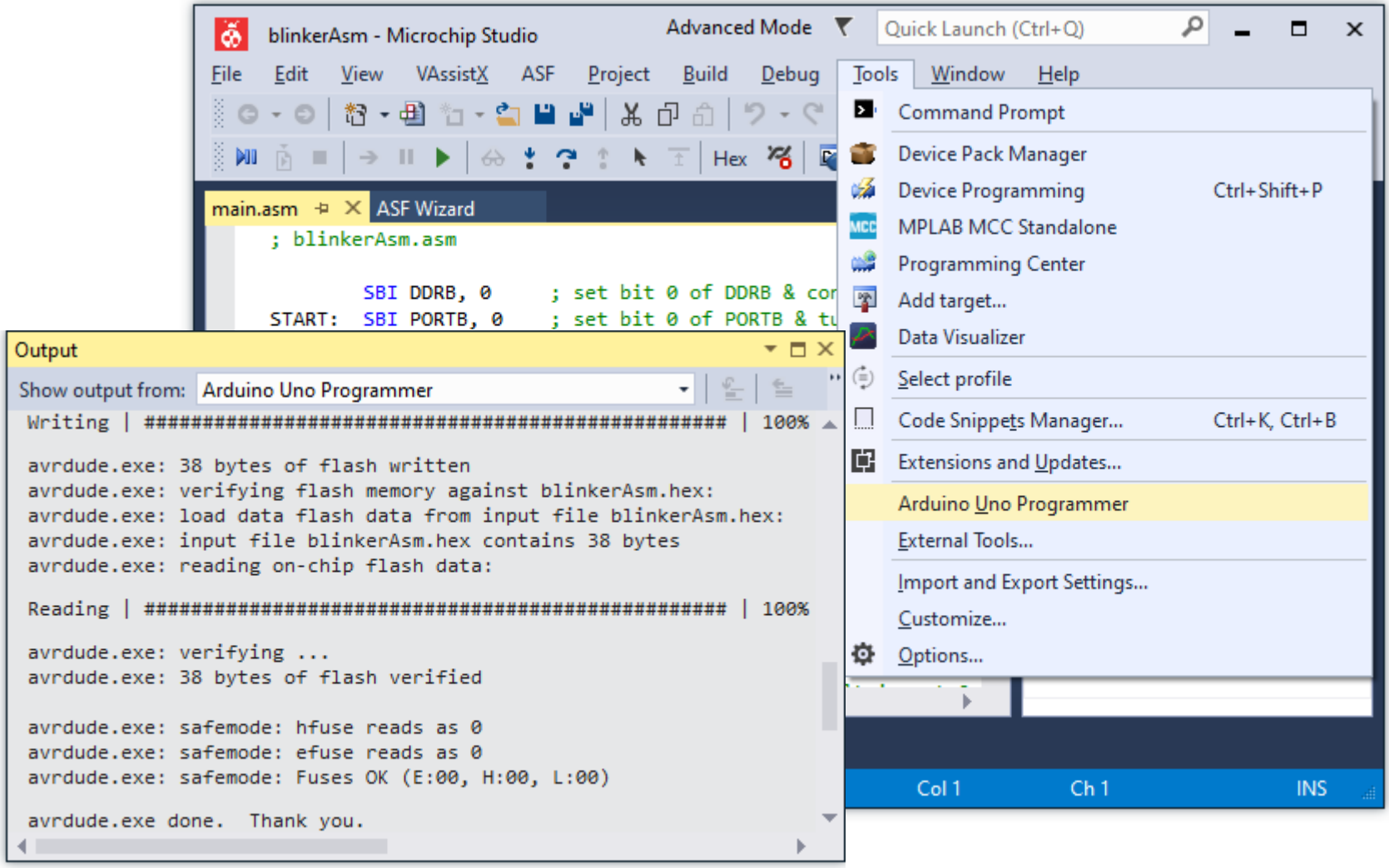
Configuring Host Application



Configuring Host Application

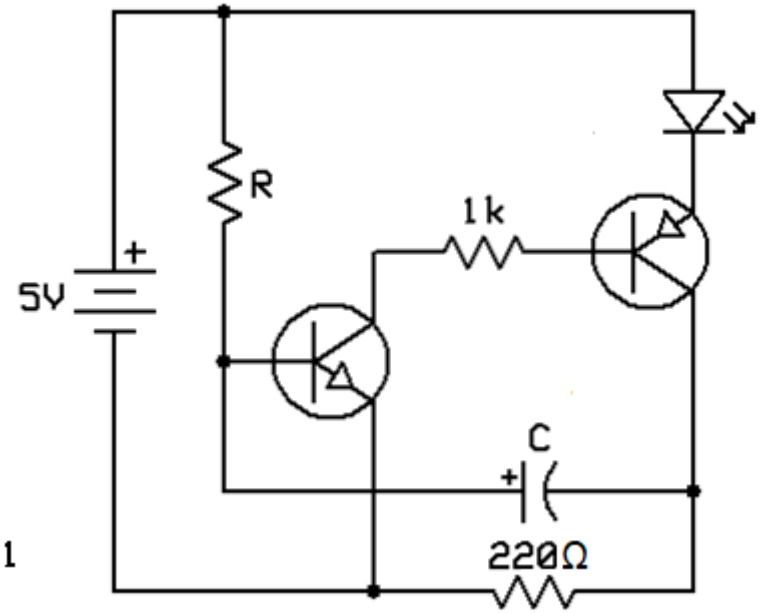
- Settings
  - Title - menu item for Atmel Studio's Tools menu
    ```
    Arduino &Uno Programmer
    ```
  - Command - path to AVRDUDE
    ```
    Drive:\...\avrdude.exe
    ```
  - Arguments - arguments passed to AVRDUDE
    ```
    -C "Drive:\...\avrdude.conf" -v -v -p atmega328p
    -c arduino -P Port -b 115200
    -U flash:w:"$(TargetName).hex":i
    ```
  - Initial Directory - directory to read the HEX file
    ```
    $(TargetDir)
    ```
  - Use Output window - whether to display processing information in output window

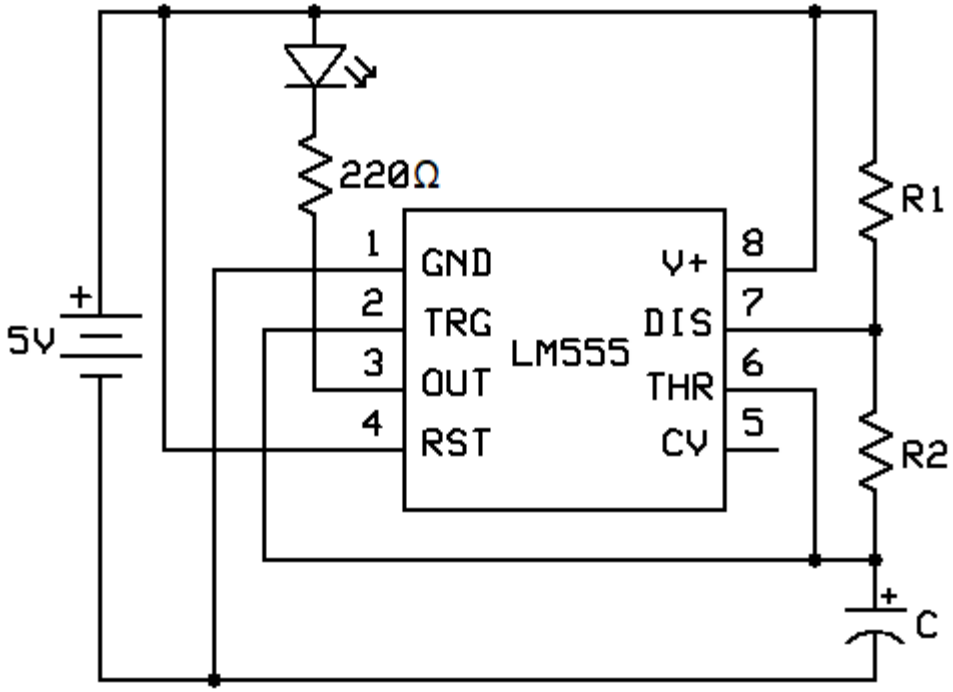
Uploading HEX File



Hardwired Alternatives

- Using Transistors
- Using an IC
- Problems?







Modified Assembly Program

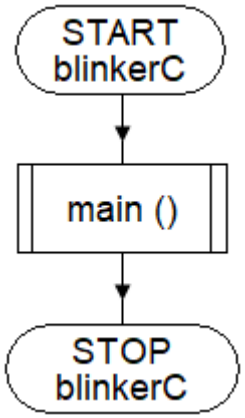
```
1   ; blinkerAsm.asm
2
3           SBI DDRB, 0     ; set bit 0 of DDRB & configure PB0 as an output
4   START:  SBI PORTB, 0    ; set bit 0 of PORTB & turn on LED
5           CALL DELAY      ; call DELAY subroutine
6           CBI PORTB, 0    ; clear bit 0 of PORTB & turn off LED
7           CALL DELAY      ; call DELAY subroutine
8           JMP START       ; jump to START
9
10  DELAY:  LDI R16, 16     ; load 16 to R16
11  LOOP1:  LDI R17, 255    ; load 255 to R17
12  LOOP2:  LDI R18, 255    ; load 255 to R18
13  LOOP3:  DEC R18         ; decrement R18 by 1
14          BRNE LOOP3      ; branch to LOOP3 if result is not 0
15          DEC R17         ; decrement R17 by 1
16          BRNE LOOP2      ; branch to LOOP2 if result is not 0
17          DEC R16         ; decrement R16 by 1
18          BRNE LOOP1      ; branch to LOOP1 if result is not 0
19          RET             ; return to calling point
```

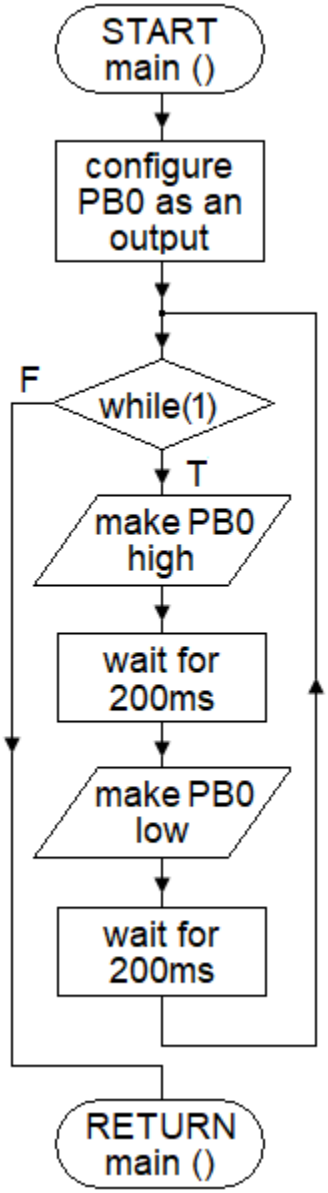
System 2

blinkerC - Blinks an LED every 200 milliseconds using C

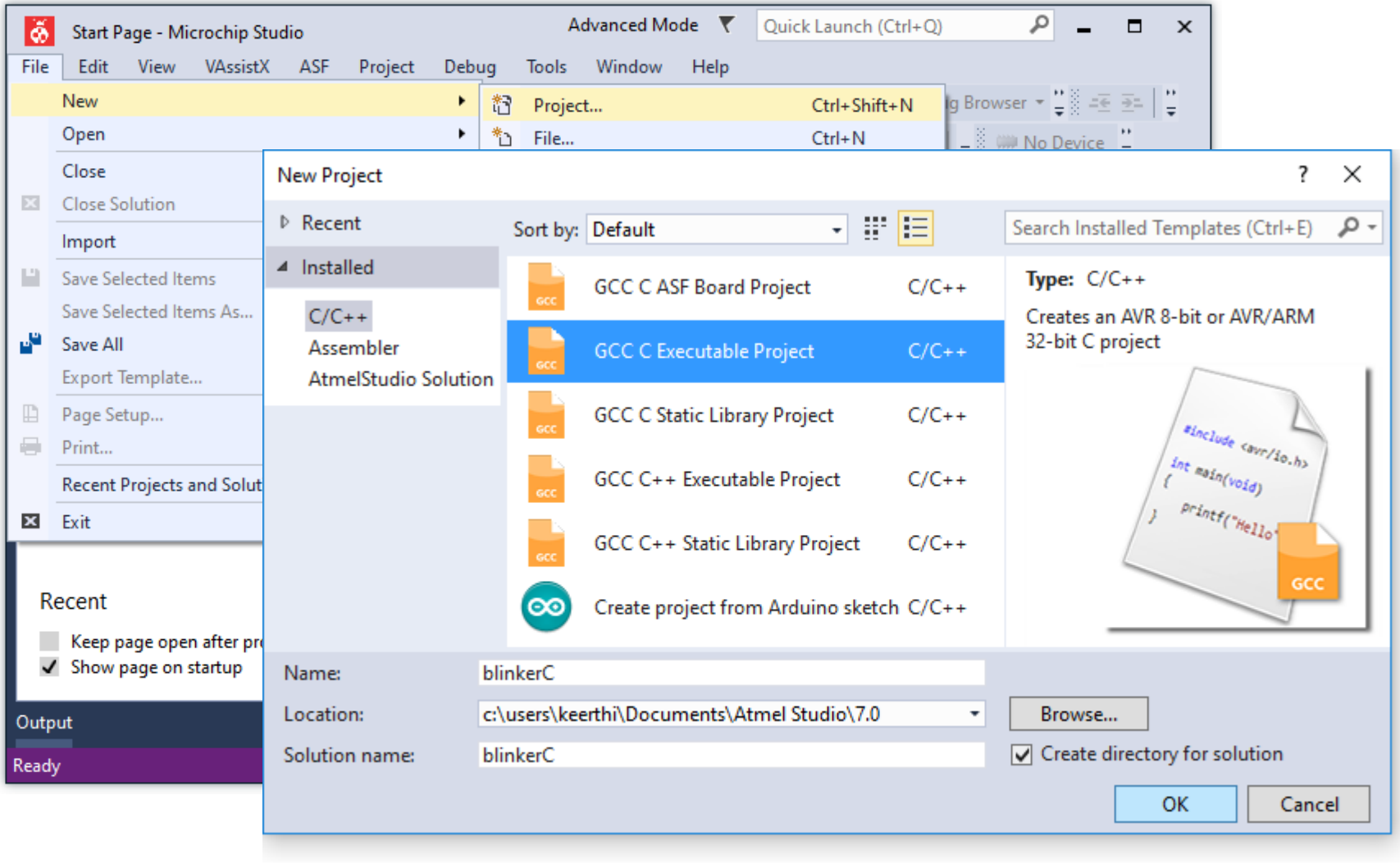
Firmware

- Algorithm





Creating a C Project



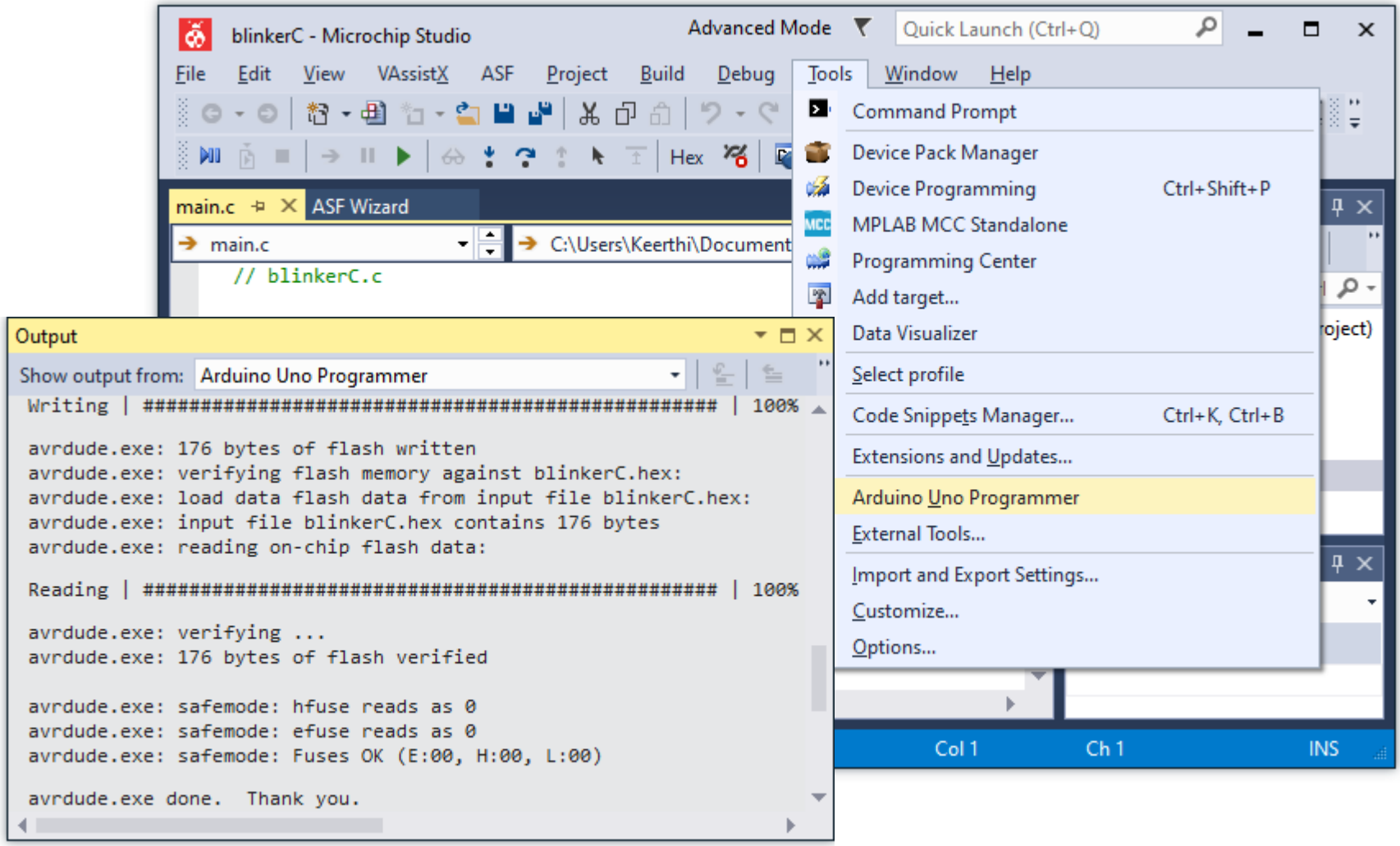
C Program

```
1   // blinkerC.c
2
3   #include <avr/io.h>          // include IO definitions
4   #define F_CPU 16000000UL     // define CPU clock speed for delay.h
5   #include <util/delay.h>      // include definition of _delay_ms( )
6
7   int main(void)
8   {
9     DDRB = 0b00000001;         // set bit 0 of DDRB & configure PB0 as an output
10    while(1)
11    {
12      PORTB = 0b00000001;      // set bit 0 of PORTB and turn on LED
13      _delay_ms(200);          // wait for 200ms
14      PORTB = 0b00000000;      // clear bit 0 of PORTB and turn off LED
15      _delay_ms(200);          // wait for 200ms
16    }
17    return 0;
18  }
```

Using Bit-wise Logical Operators

```
1   // blinkerC.c
2
3   #include <avr/io.h>                 // include IO definitions
4   #define F_CPU 16000000UL            // define CPU clock speed for delay.h
5   #include <util/delay.h>             // include definition of _delay_ms( )
6
7   int main(void)
8   {
9     DDRB = DDRB | 0b00000001;         // set bit 0 of DDRB & configure PB0 as an output
10    while(1)
11    {
12      PORTB = PORTB | 0b00000001;     // set bit 0 of PORTB and turn on LED
13      _delay_ms(200);                 // wait for 200ms
14      PORTB = PORTB & 0b11111110;     // clear bit 0 of PORTB and turn off LED
15      _delay_ms(200);                 // wait for 200ms
16    }
17    return 0;
18  }
```

Uploading HEX File

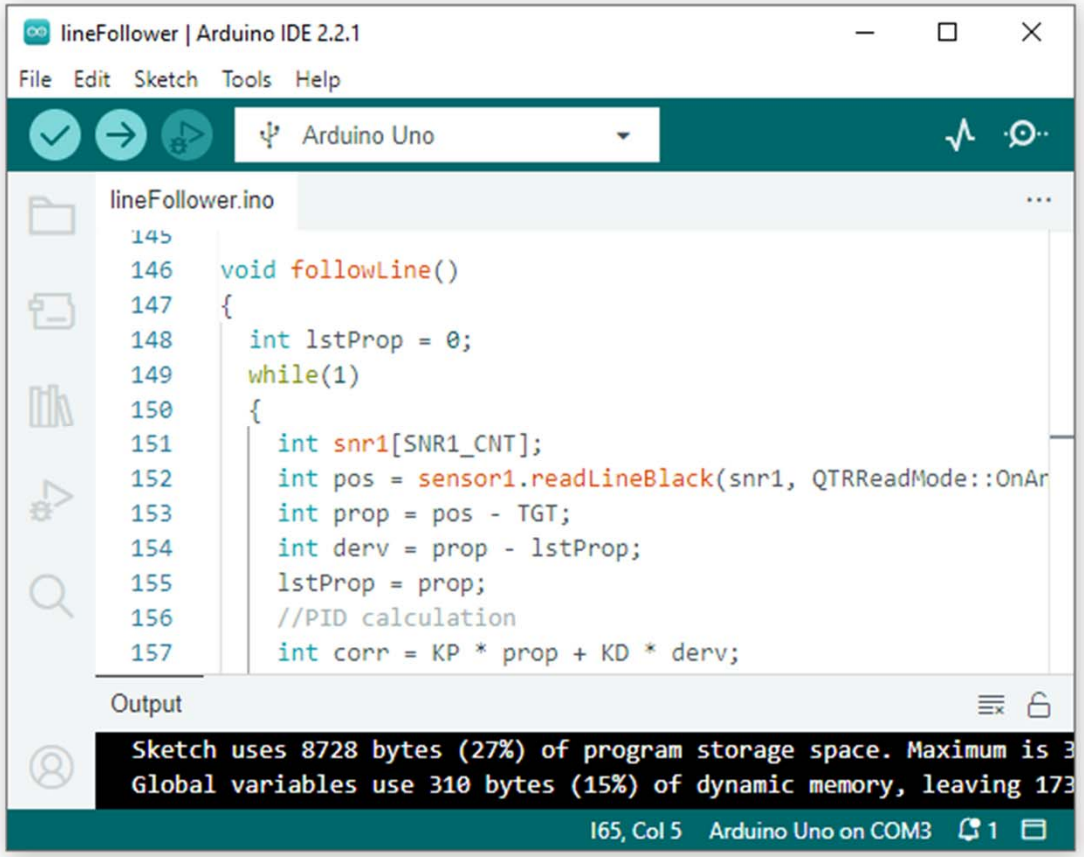


System 3

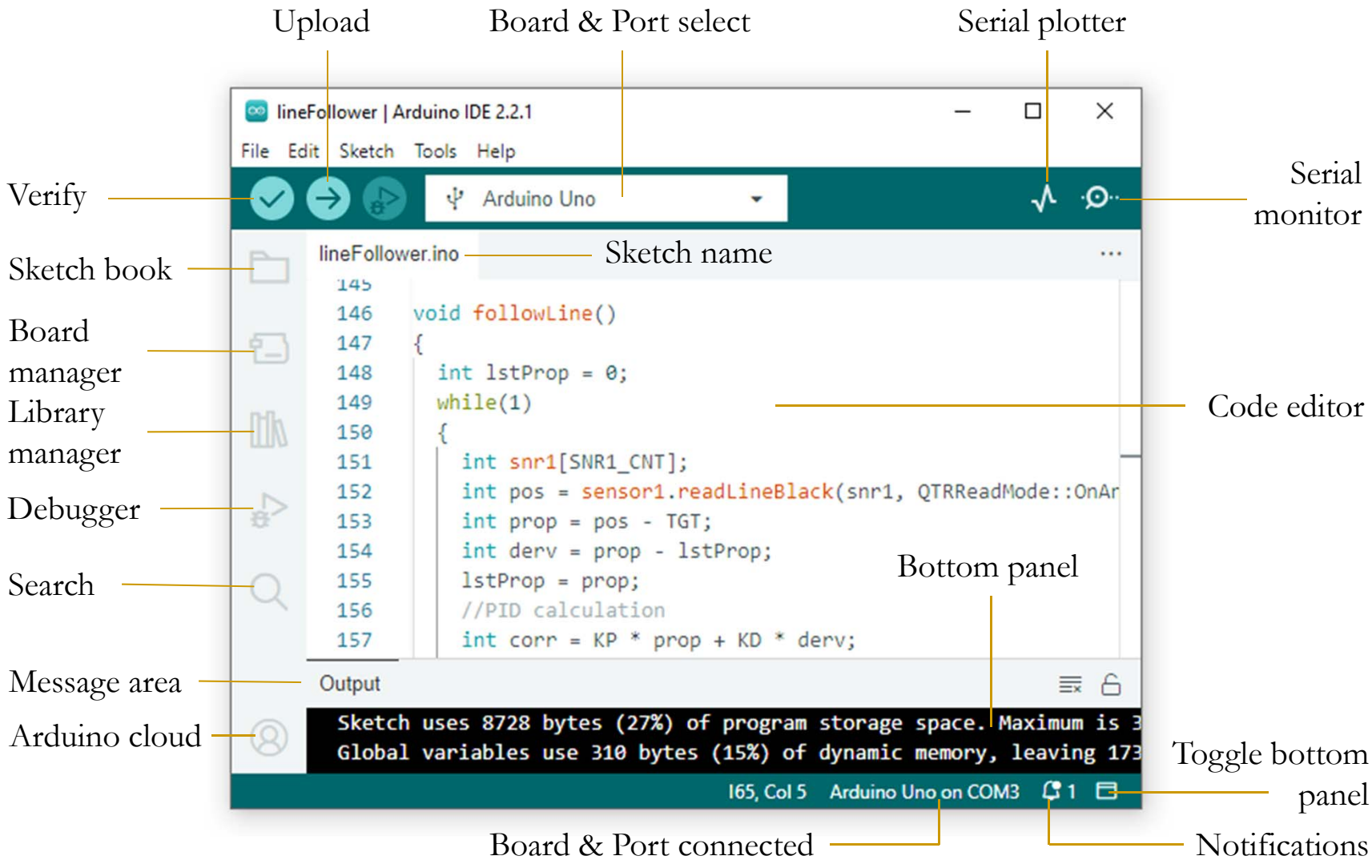
blinker - Blinks an LED every 200 millisecond using Arduino

Arduino IDE

- Integrates a code editor, compiler, debugger, uploader, text terminal and other tools
- Freely available to download from https://www.arduino.cc/en/software

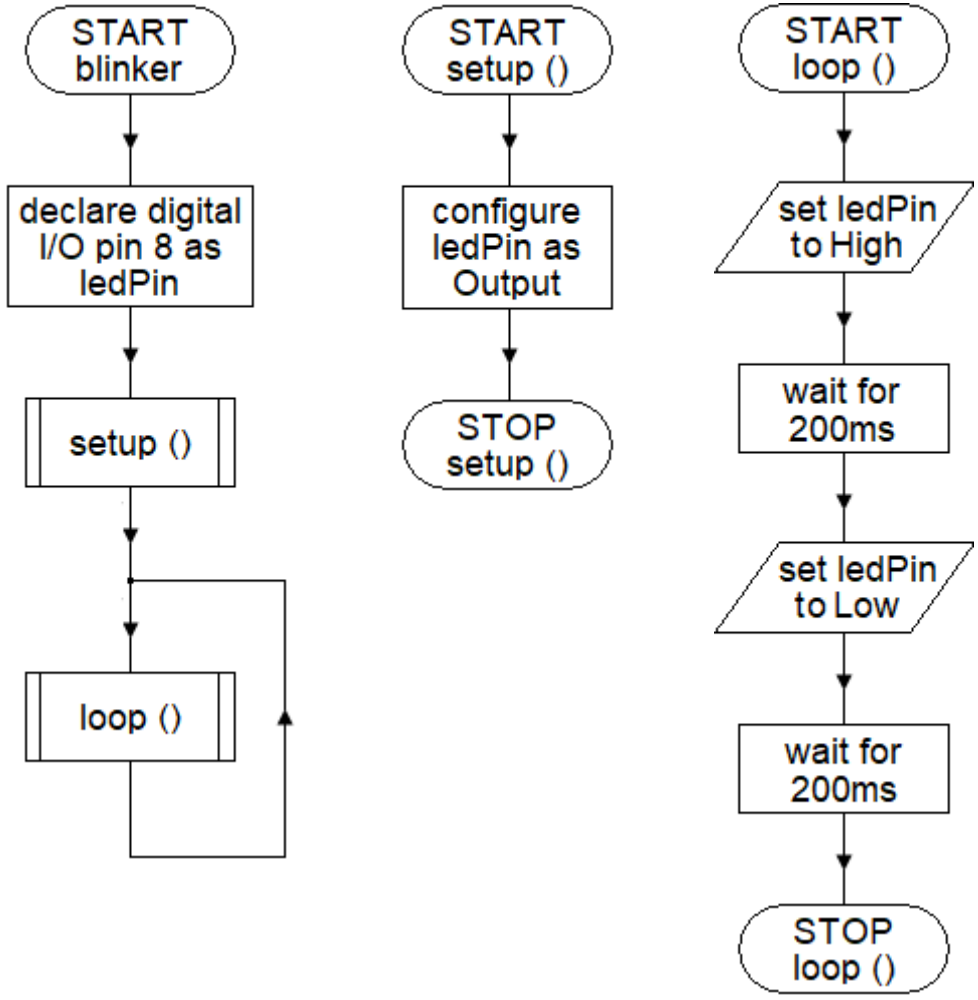


Arduino IDE


Upload
Board & Port select
Serial plotter
lineFollower | Arduino IDE 2.2.1
File Edit Sketch Tools Help
Arduino Uno
Verify
Serial monitor
Sketch book
lineFollower.ino
Sketch name
Board manager
Library manager
Code editor
Debugger
Search
Bottom panel
Message area
Output
Arduino cloud
Sketch uses 8728 bytes (27%) of program storage space. Maximum is 3
Global variables use 310 bytes (15%) of dynamic memory, leaving 173
Toggle bottom panel
Board & Port connected
Notifications

Firmware

- Algorithm
  - setup function
  - loop function



Firmware

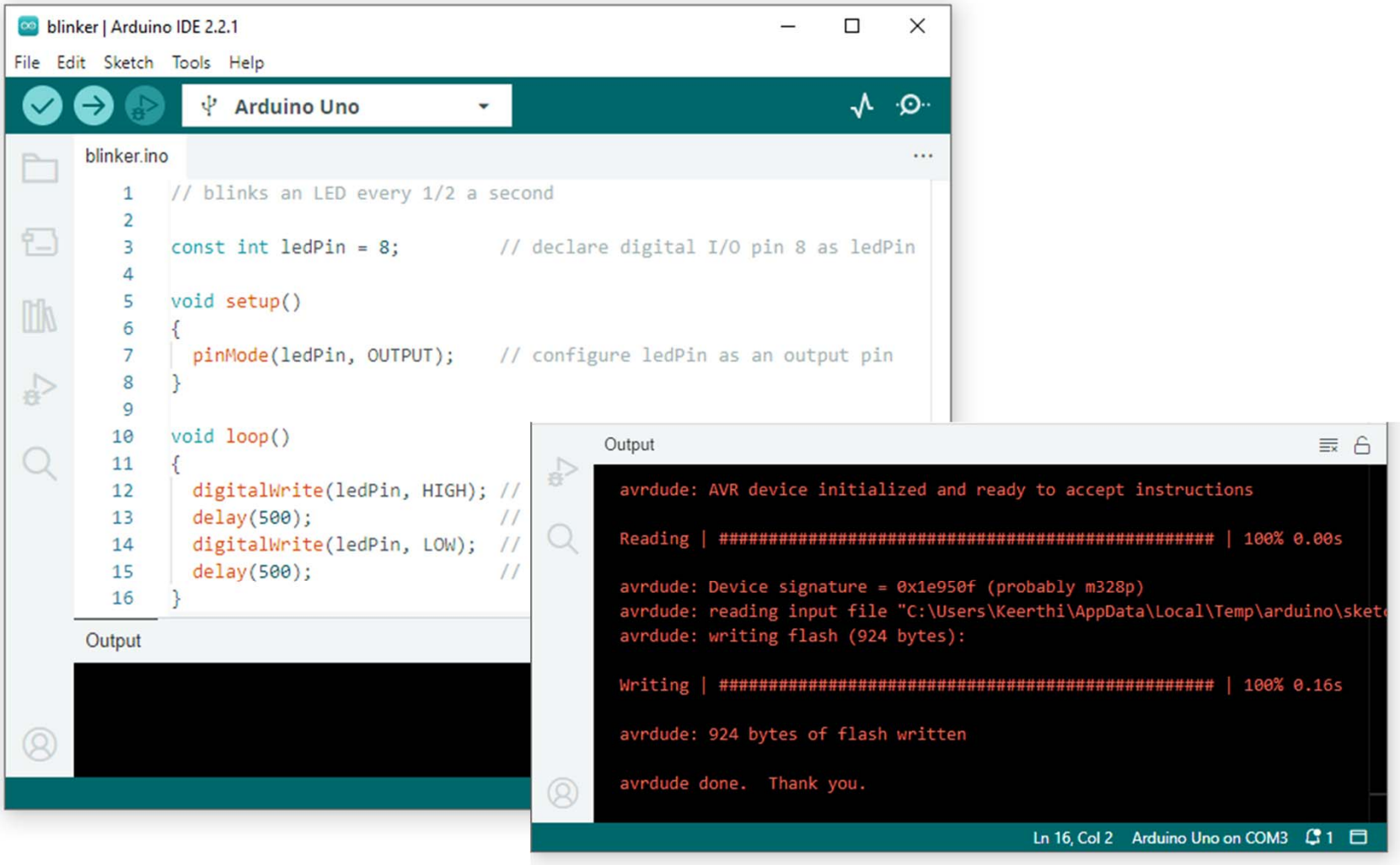
- Source Code

```
// blinker.ino
const int ledPin = 8;          // declare digital I/O pin 8 as ledPin

void setup()
{
  pinMode(ledPin, OUTPUT);     // configure ledPin as output
}

void loop()
{
  digitalWrite(ledPin, HIGH); // set ledPin to High
  delay(200);                  // wait for 200ms
  digitalWrite(ledPin, LOW);   // set ledPin to Low
  delay(200);                  // wait for 200ms
}
```

Compiling and Uploading

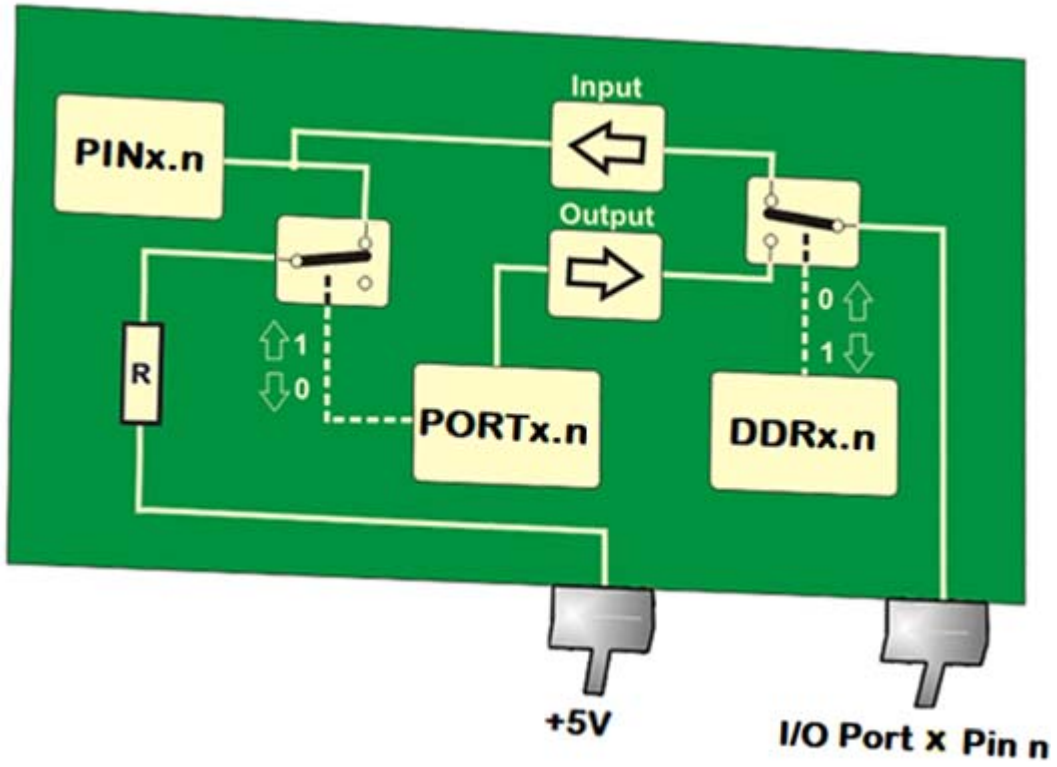


Comparison

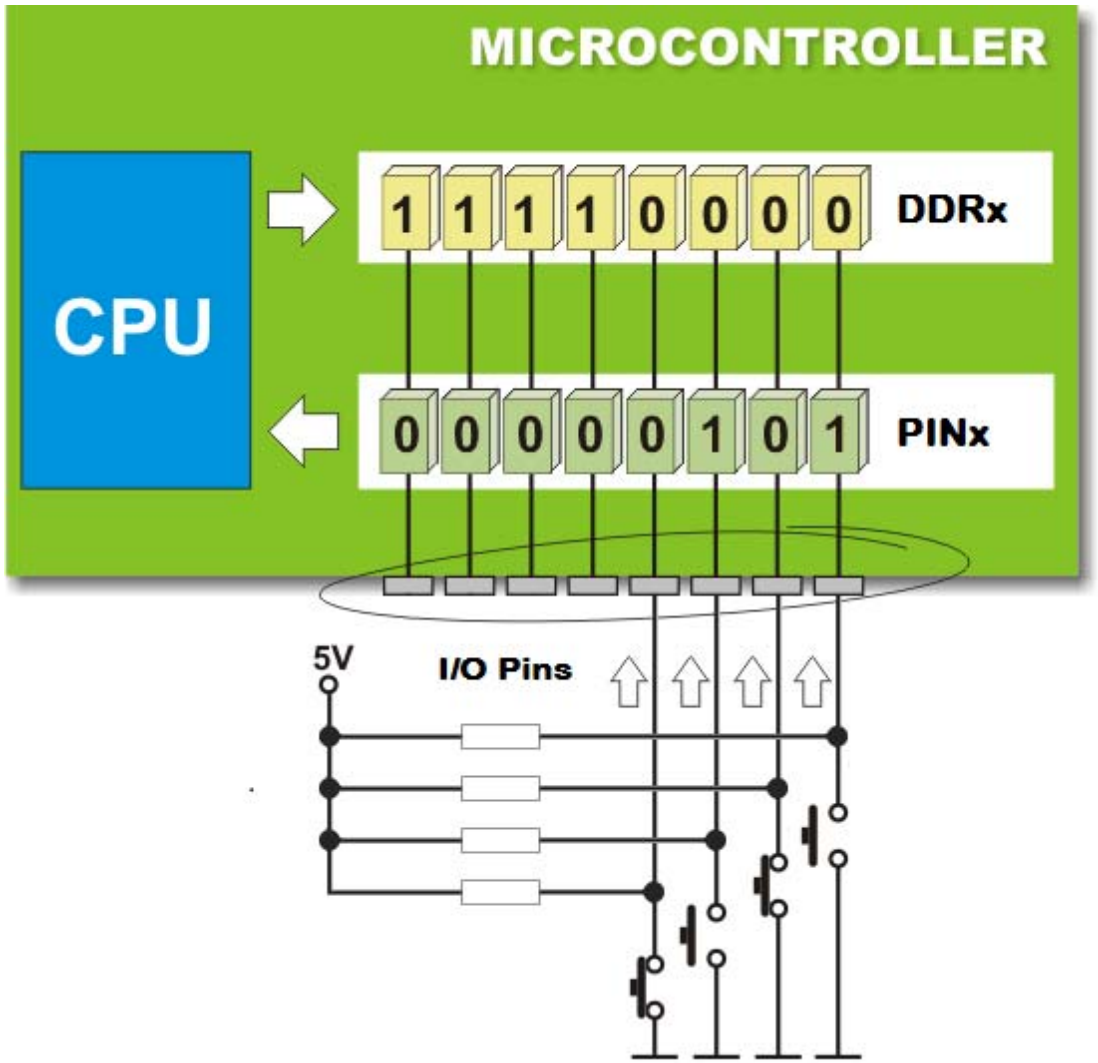
- Assembly
  - Highly efficient, difficult to code, not portable
- C
  - Efficient, easier to code, libraries available, portable
  - Data type misuse possible
- Arduino
  - Very easier to code, libraries available
  - Inefficient, not portable
  - How come Arduino becomes inefficient, if it also uses GCC and AVRDUDE?

I/O Registers

- Port INput register (PIN)
  - Used to read the state present at port pins
  - Set the bits of PORT register to activate internal pull-up resistors



DDR and PIN Registers



System 4

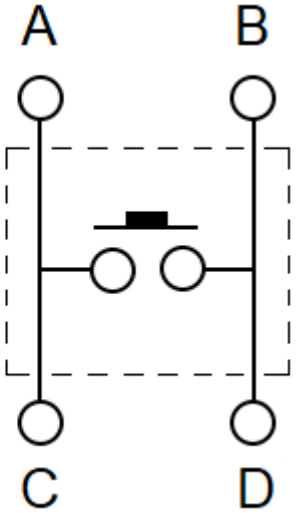
dualRateBlinker - Blinks an LED every 500ms if no input, otherwise blinks it every 200ms

Micro Pushbutton Switch

- Connects two points in a circuit when pressed
- Normally open and closed when pressed

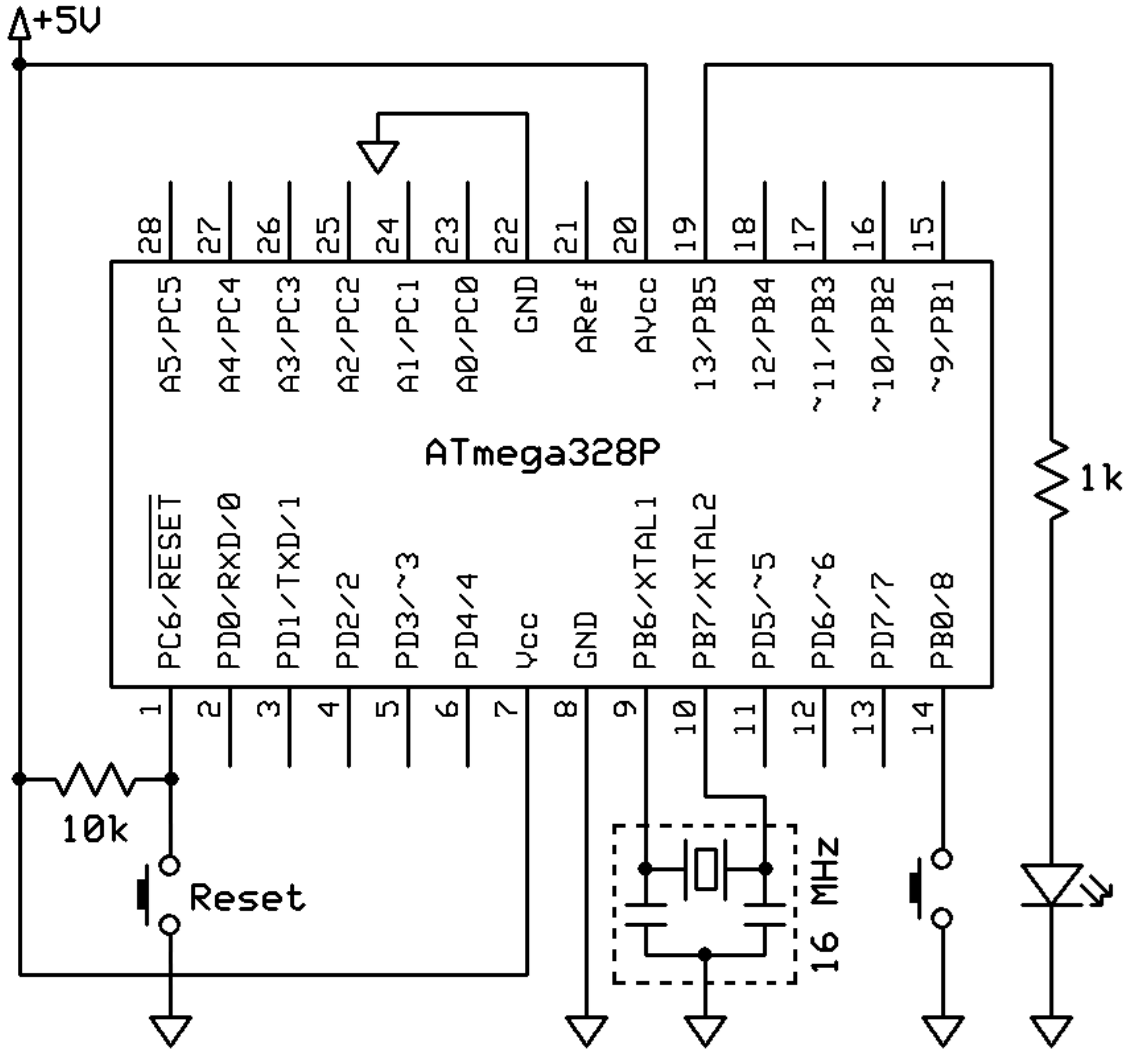






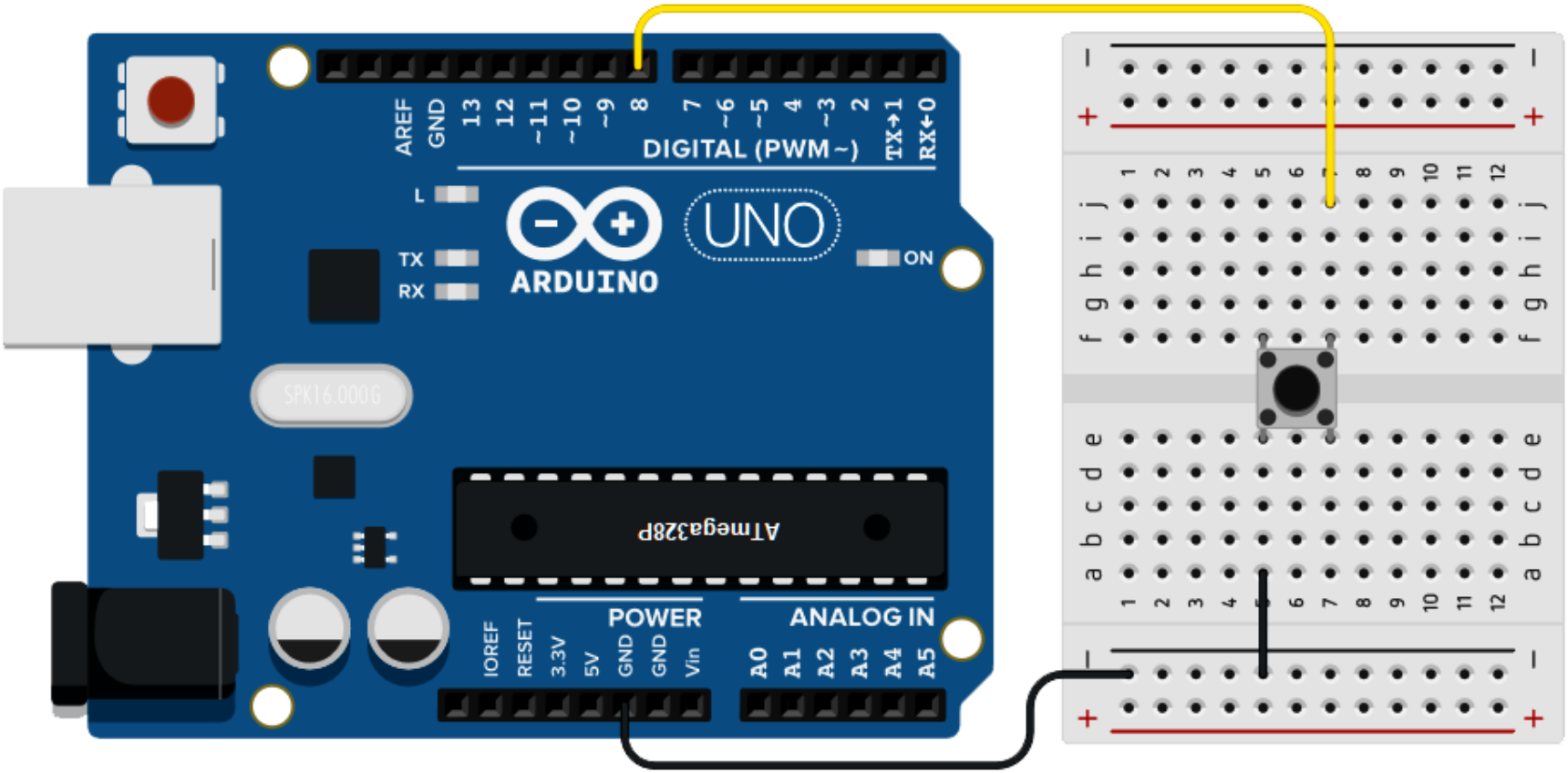
Hardware

- Schematic Diagram



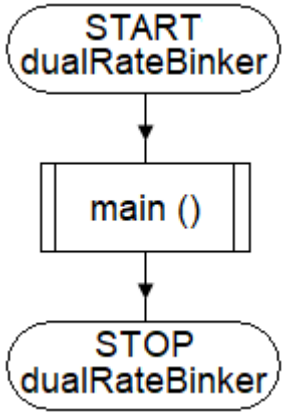
Hardware

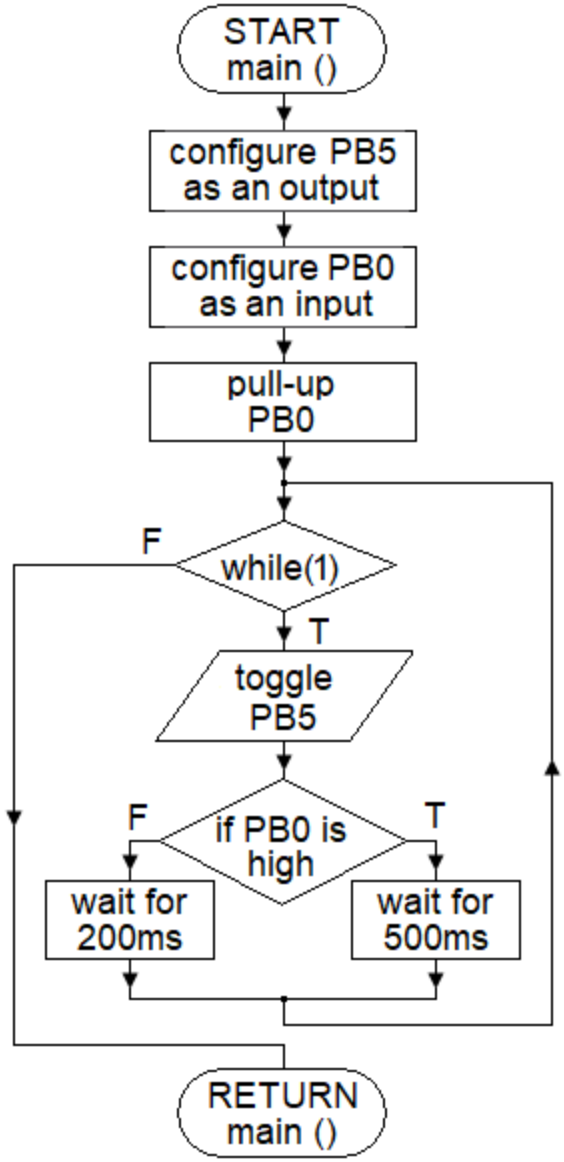
- Wiring Diagram



Firmware

- Algorithm





C Program

```
1   // dualRateBlinkerC.c
2
3   #include <avr/io.h>
4   #define F_CPU 16000000UL                // define CPU clock speed for delay.h
5   #include <util/delay.h>
6
7   int main(void)
8   {
9       DDRB = DDRB | 0b00100000;           // configure PB5 as an output
10      DDRB = DDRB & 0b11111110;           // configure PB0 as an input
11      PORTB = PORTB | 0b00000001;         // pull-up PB0
12      while (1)
13      {
14          PORTB = PORTB ^ 0b00100000;     // toggle PB5
15          if (PINB & 0b00000001)          // if PB0 is high
16              _delay_ms(500);             // delay 500ms
17          else
18              _delay_ms(200);             // otherwise delay 200ms
19      }
20      return 0;
21  }
```

Using Bit-wise Shift Operators

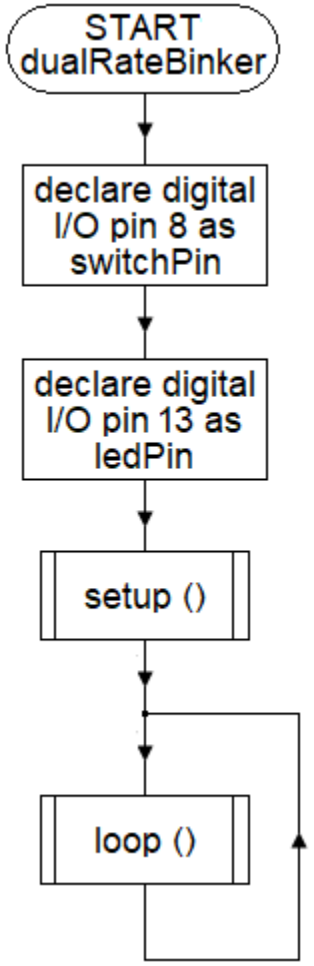
```
1    // dualRateBlinkerC.c
2
3    #include <avr/io.h>
4    #define F_CPU 16000000UL                  // define CPU clock speed for delay.h
5    #include <util/delay.h>
6
7    int main(void)
8    {
9        DDRB = DDRB | (1<<5);                 // configure PB5 as an output
10       DDRB = DDRB & ~(1<<0);                // configure PB0 as an input
11       PORTB = PORTB | (1<<0);               // pull-up PB0
12       while (1)
13       {
14           PORTB = PORTB ^ (1<<5);           // toggle PB5
15           if (PINB & (1<<0))                // if PB0 is high
16               _delay_ms(500);               // delay 500ms
17           else
18               _delay_ms(200);               // otherwise delay 200ms
19       }
20       return 0;
21   }
```

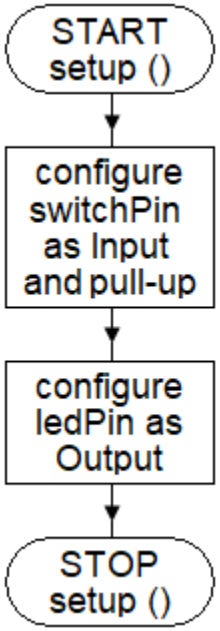
Using Labeled Constants

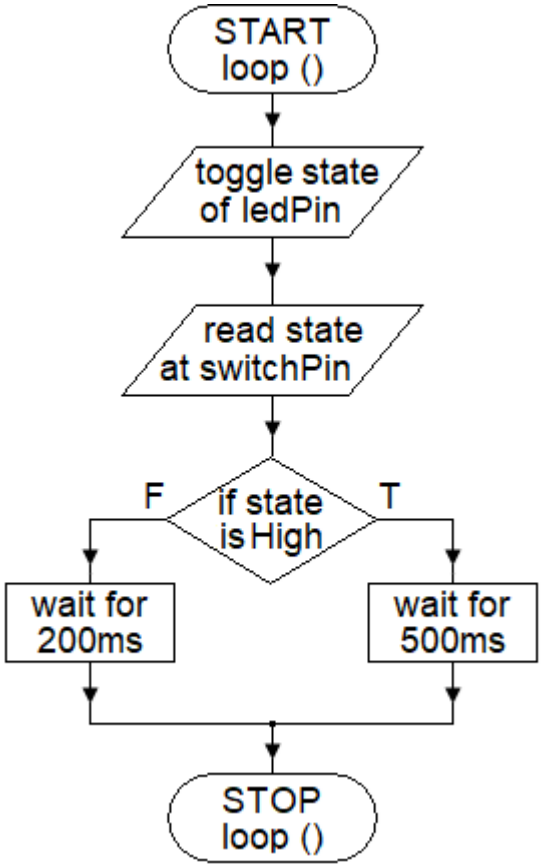
```
1   // dualRateBlinkerC.c
2
3   #include <avr/io.h>
4   #define F_CPU 16000000UL                // define CPU clock speed for delay.h
5   #include <util/delay.h>
6
7   int main(void)
8   {
9       DDRB = DDRB | (1<<DDB5);            // configure PB5 as an output
10      DDRB = DDRB & ~(1<<DDB0);           // configure PB0 as an input
11      PORTB = PORTB | (1<<PORTB0);        // pull-up PB0
12      while (1)
13      {
14          PORTB = PORTB ^ (1<<PORTB5);    // toggle PB5
15          if (PINB & (1<<PINB0))          // if PB0 is high
16              _delay_ms(500);             // delay 500ms
17          else
18              _delay_ms(200);             // otherwise delay 200ms
19      }
20      return 0;
21  }
```

Firmware

- Algorithm for Arduino







Firmware

- Source Code (using Arduino Wrapper Functions)

```
// dualRateBlinker.ino

const int switchPin = 8;              // declare digital I/O pin 8 as switchPin
const int ledPin = 13;                // declare digital I/O pin 13 as ledPin

boolean state = false;                // declare Boolean variable & initialize with false

void setup()
{
  pinMode(switchPin, INPUT_PULLUP); // configure switchPin as input and pull-up
  pinMode(ledPin, OUTPUT);          // configure ledPin as output
}

void loop()
{
  state = !state;                     // toggle value of Boolean variable
  digitalWrite(ledPin, state);        // toggle state of ledPin
  if (digitalRead(switchPin))         // read state at switchPin and if it is high
    delay(500);                       // wait for 500ms
  else
    delay(200);                       // otherwise wait for 200ms
}
```

Questions